

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ



BÁO CÁO ĐỒ ÁN THỰC TẬP KỸ SƯ
ĐỀ TÀI: THIẾT KẾ LỖI IP TĂNG TỐC CHO BỘ SOFTMAX ATTENTION TRÊN KIT
KV260
GVHD: TS. TRƯƠNG QUANG VINH
VERSION: 03

HỌ VÀ TÊN	MÃ SỐ SINH VIÊN
HOÀNG TIẾN PHÁT	2212506

TP. HỒ CHÍ MINH

MỤC LỤC

MỤC LỤC HÌNH ẢNH	3
MỤC LỤC BẢNG	4
I. TỔNG QUAN.....	5
1. Kiến trúc hệ thống	5
II. CẤU TRÚC IP MATMUL LINEAR	6
1. Cấu trúc IP AXI Linear	6
1.1. FSM IP AXI Linear.....	6
1.2. Giải thuật thực thi FSM Linear.....	7
1.3. Module matmul_ip.....	11
1.4. Module pe_unit – Khối pe thực hiện nhân cộng tích lũy	16
III. CẤU TRÚC IP SOFTMAX SCORE.....	17
1. Cấu trúc IP AXI Softmax	17
1.1. FSM IP Softmax	18
1.2. Giải thuật thực thi FSM Softmax score.....	21
1.3. Module reciprocal_divider.....	24
IV. MÔ PHỎNG VIVADO	25
1. Cấu trúc thực thi.....	25
2. Cấu trúc mô phỏng	28
3. Kết quả mô phỏng.....	31
V. THỰC HIỆN TRÊN KV260	31
1. Cấu trúc thực thi.....	31
2. Thông số thiết lập	35
3. Kết quả thực thi.....	35
3.1. Kết quả thực thi với phương pháp Baremetal	35
3.2. Kết quả thực thi với Ubuntu Linux.....	36
VI. TÀI LIỆU THAM KHẢO	37
VII. PHỤ LỤC.....	37

MỤC LỤC HÌNH ẢNH

Hình 1: Sơ đồ kiến trúc hệ thống	5
Hình 2: Sơ đồ khối cấu trúc IP Linear.....	6
Hình 3: Sơ đồ FSM Linear	7
Hình 4: Lưu đồ giải thuật của IP BRAM k ram.....	8
Hình 5: Lưu đồ giải thuật state Preload mac	8
Hình 6: Lưu đồ giải thuật 1 của state COMPUTE.....	9
Hình 7: Lưu đồ giải thuật 2 của state COMPUTE.....	10
Hình 8: Sơ đồ cấu trúc nạp dữ liệu cho PE trong state COMPUTE với N TILES = 1	12
Hình 9: Lưu đồ giải thuật của submodule matmul_ip trong 2 state PRELOAD MAC và COMPUTE của module linear trường hợp N PE = N COLS.....	13
Hình 10: Sơ đồ cấu trúc nạp dữ liệu cho PE trong state COMPUTE với N TILES > 1	14
Hình 11: Lưu đồ giải thuật của submodule matmul_ip trong 2 state PRELOAD MAC và COMPUTE của module linear trường hợp N PE < N COLS.....	15
Hình 12: Lưu đồ giải thuật phần mềm của module pe_unit	16
Hình 13: Sơ đồ khối cấu trúc IP Softmax.....	18
Hình 14: FSM IP Softmax	18
Hình 15: Lưu đồ giải thuật của FSM Load Row	21
Hình 16: Lưu đồ giải thuật của FSM state Find Max	22
Hình 17: Lưu đồ giải thuật của FSM Exp Sum	23
Hình 18: Lưu đồ giải thuật FSM của ST DIV ISSUE và ST DIV DRAIN.....	23
Hình 19: Lưu đồ giải thuật thực thi module reciprocal_divider	24
Hình 20: Lưu đồ giải thuật của FSM SERIALIZER.....	25
Hình 21: Sơ đồ cấu trúc phần cứng mô phỏng trên Vivado	26
Hình 22: Block Design mô phỏng Vivado	26
Hình 23: Sơ đồ PS lai PL cấu trúc mô phỏng.....	28
Hình 24: Sơ đồ khối PL thực hiện testbench	29
Hình 25: Kết quả tcl của tb và giá trị rtl softmax được lưu trong mem file	31
Hình 26: Softmax data được push vào BRAM DST từ DMA S2MM.....	31
Hình 27: Sơ đồ khối cấu trúc phần cứng thực thi trên KV260	32
Hình 28: Cấu trúc Block Design.....	32
Hình 29: Sơ đồ khối luồng thực thi bằng Baremetal.....	34
Hình 30: Sơ đồ khối luồng thực thi bằng linux thông qua SD card.....	34
Hình 31: Kết quả tính toán attention score trên kv260	35
Hình 32: Kết quả tính toán softmax score trên kv260	36
Hình 33: Kết quả push dự án bằng linux.....	37

MỤC LỤC BẢNG

Table 1: Parameter module matmul_ip	11
Table 2: Localparameter module matmul_ip	11
Table 3: Parameter module pe_unit	16
Table 4: Localparameter module pe_unit	16
Table 5: Ý nghĩa các tín hiệu đầu vào của module pe_unit trong phase PRELOAD MAC của module linear	17
Table 6: Ý nghĩa các tín hiệu đầu vào của module pe_unit trong phase COMPUTE của module linear.....	17
Table 7: Thông tin các IP được sử dụng trong testbench	27
Table 8: Địa chỉ các IP được sử dụng trong testbench	27
Table 9: Thông số chuẩn hóa khi mô phỏng testbench	31
Table 10: Thông tin các IP được sử dụng khi thực thi trên KV260	33
Table 11: Địa chỉ các IP được sử dụng khi thực thi trên KV260	33
Table 12: Thông số chuẩn hóa được sử dụng khi thực thi trên KV260	35

I. TỔNG QUAN

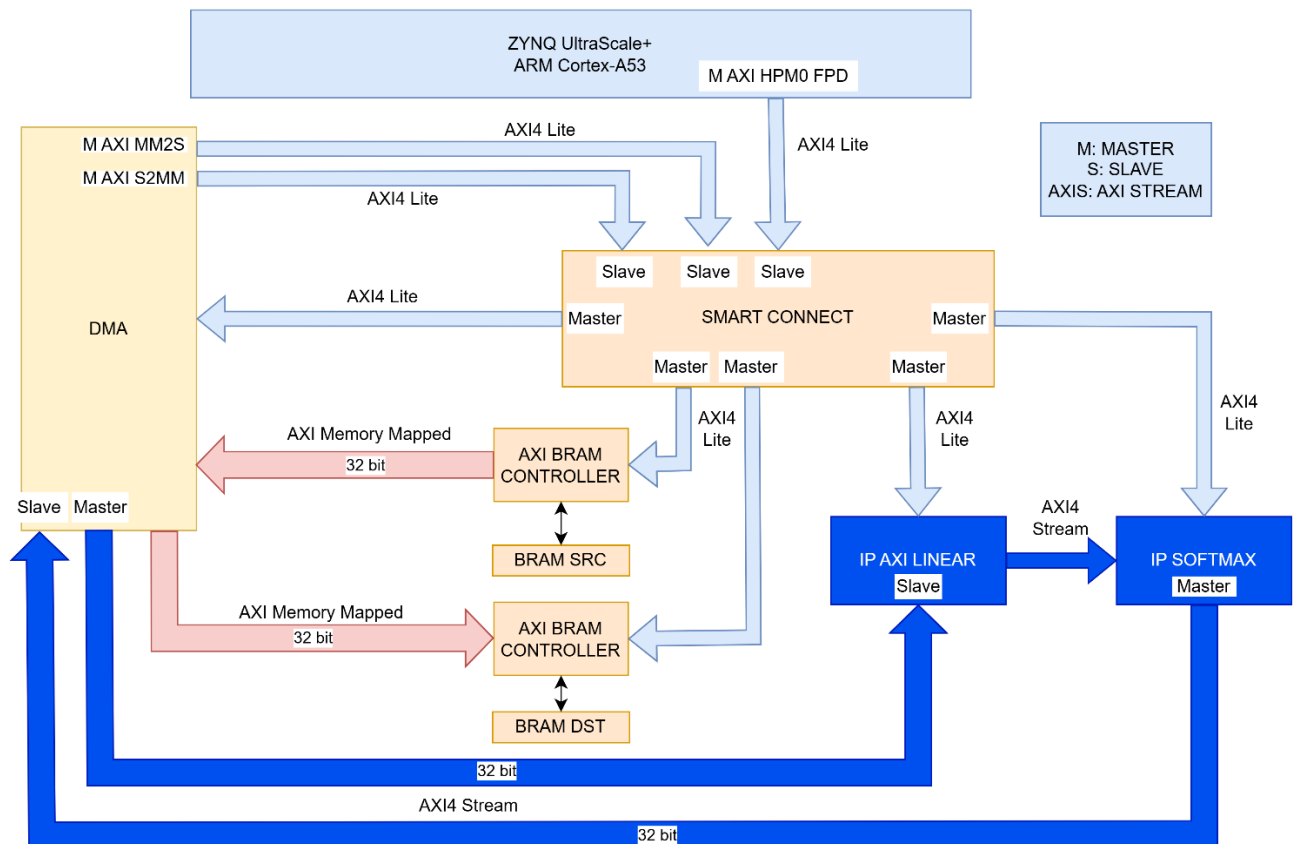
Hệ thống tính Softmax($Q \times K^T$) — 2 giai đoạn đầu của Scaled Dot-Product Attention — trên phần cứng Zynq UltraScale+ (KV260), điều khiển bằng bare-metal C qua Vitis.

- IP Linear: tính $S = Q \times K^T$.
- IP Softmax: nhận S , tính softmax theo từng hàng, trả về attention weight (Q1.15 unsigned).
- Kết nối 2 IP qua DMA (không stream trực tiếp IP-to-IP): linear ghi kết quả S ra BRAM qua S2MM, sau đó softmax đọc lại từ BRAM qua MM2S riêng — 2 IP không share cùng 1 kênh AXI-Stream.
- Điều khiển: mỗi IP có control-plane AXI4-Lite riêng (start/status), data-plane qua axi_dma_0 (1 kênh MM2S + 1 kênh S2MM dùng chung cho cả 2 IP, tuần tự).

Đã tích hợp và test PASS trên KV260 với cấu hình tham số:

$$SEQ_LEN = D_HEAD = N_PE = DATA_WIDTH = 16$$

1. Kiến trúc hệ thống



Hình 1: Sơ đồ kiến trúc hệ thống

- bram_ctrl_src: chứa K data và Q data (preload bởi PS trước khi chạy).
- bram_ctrl_dst: ghi kết quả output softmax (đọc S từ chính bram_dst, ghi weight trở lại bram_dst). Xem chi tiết thứ tự ở mục 6.
- Interconnect: AXI SmartConnect (1 clock domain, không CDC nội bộ).

II. CẤU TRÚC IP MATMUL LINEAR

Bài nghiên cứu đã cải tiến cấu trúc IP so với cấu trúc cũ là dual axi lite dùng để control và push data vào IP Linear thì cấu trúc mới thực hiện như sau:

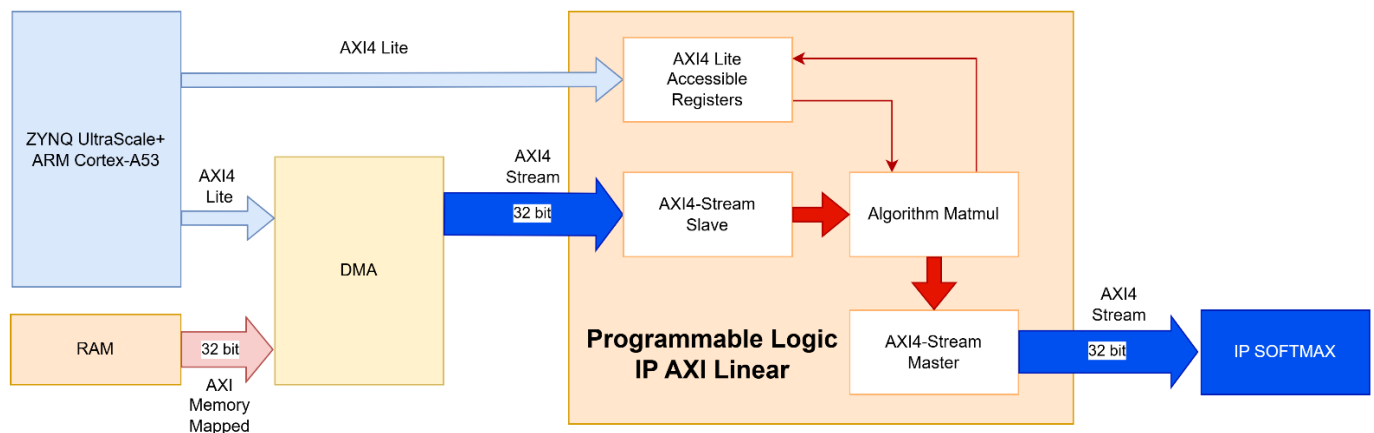
- Interface gồm 1 axi lite control status của IP Linear như push start signal hoặc read done, busy signal, và 1 axi stream để push data vào IP để tính matmul.
- Giảm số lượng ip BRAM từ 4 ip q, k, v, sram còn 1 ip BRAM là k ram
- Master thay cho cpu thay vì 2 axi vip thì sử dụng 1 axi vip. Ngoài ra, bài nghiên cứu đã thực hiện tích hợp DMA kết hợp với IP Linear.

Cấu trúc IP AXI Linear bao gồm 1 AXI Lite accessible register và 1 module linear giao tiếp axi stream. Module linear có module core thực hiện matmul là matmul_ip, module này thực hiện phép tính matmul theo phương pháp SIMD-Broadcast thông qua các pe, tập hợp N pe sẽ có 1 số lượng tile, nguyên lý hoạt động của các pe là module pe_unit.

1. Cấu trúc IP AXI Linear

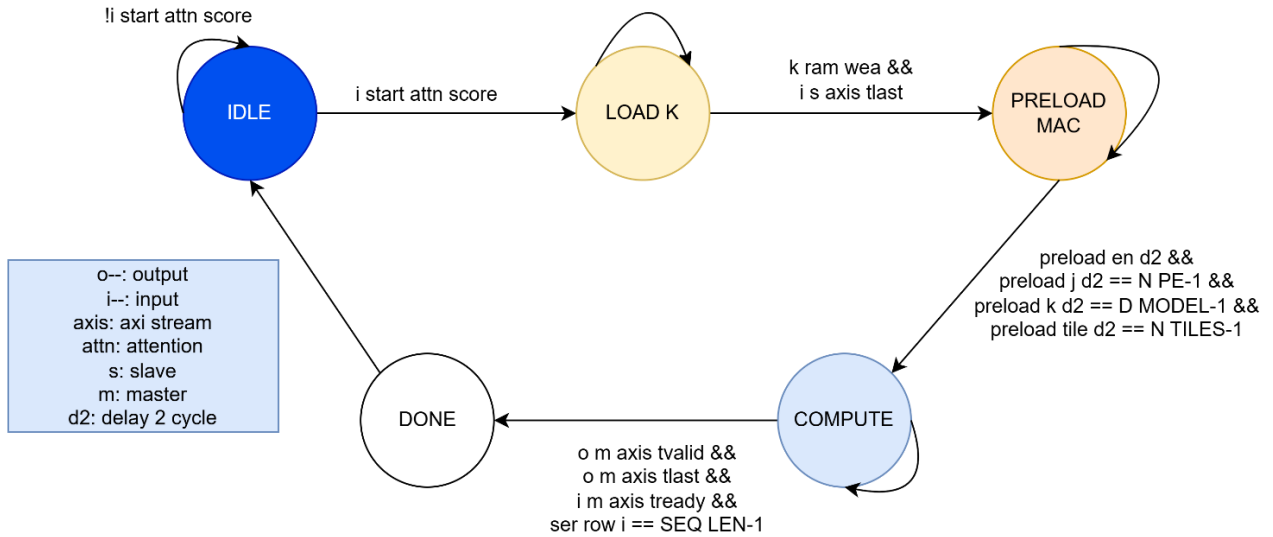
Cấu trúc IP linear thực hiện mix giữa axi lite và axi stream:

- DMA sẽ đóng vai trò master stream data K và Q từ DDR hoặc BRAM bên ngoài vào trong ip linear để thực hiện phép tính matmul và chia $\sqrt{d_{model}}$.
- Sau khi IP compute xong, thực hiện truyền AXI stream ra ngoài, đi vào S2MM của DMA để DMA ghi xuống DDR hoặc BRAM.



Hình 2: Sơ đồ khối cấu trúc IP Linear

1.1. FSM IP AXI Linear



Hình 3: Sơ đồ FSM Linear

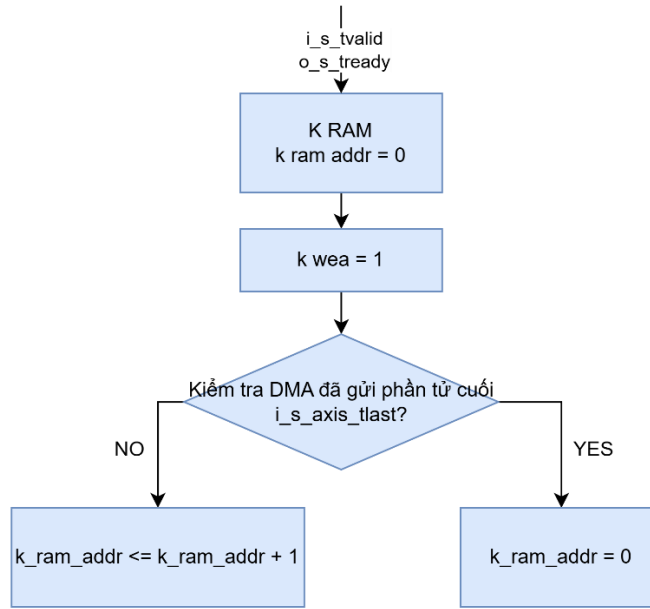
FSM của linear gồm 5 state, mỗi state có công dụng như sau:

- IDLE: trạng thái tĩnh, đợi tín hiệu start thông qua AXI Lite để chuyển sang state LOAD K.
- LOAD K: IP BRAM trong linear sẽ được load data của k ram thông qua interface AXI stream, nguồn từ DMA đọc trong DDR truyền tới. Kết thúc quá trình này khi tín hiệu tlast xuất hiện. State tiếp theo là PRELOAD MAC
- PRELOAD MAC: trạng thái preload data k ram vào các pe của module matmul, mỗi pe là 1 hàng data của k ram, số pe user có thể nhập input, nếu số pe nhỏ hơn d head (Số hàng của k ram) thì matmul sẽ sử dụng phương pháp tilling.
- COMPUTE: DMA MM2S thực hiện stream data Q vào trong các pe để tính toán, mỗi lần thực hiện stream 1 hàng Q gồm d model = 64 phần tử, các pe sẽ tính toán và cho ra cùng lúc 1 hàng của score (Nếu N PE = D HEAD), nếu N PE < D HEAD thì thực hiện phương pháp tilling đến khi tính xong 1 hàng thì xuất ra cổng stream đến DMA S2MM, phương pháp tính được áp dụng là SIMD-style inner product.

1.2. Giải thuật thực thi FSM Linear

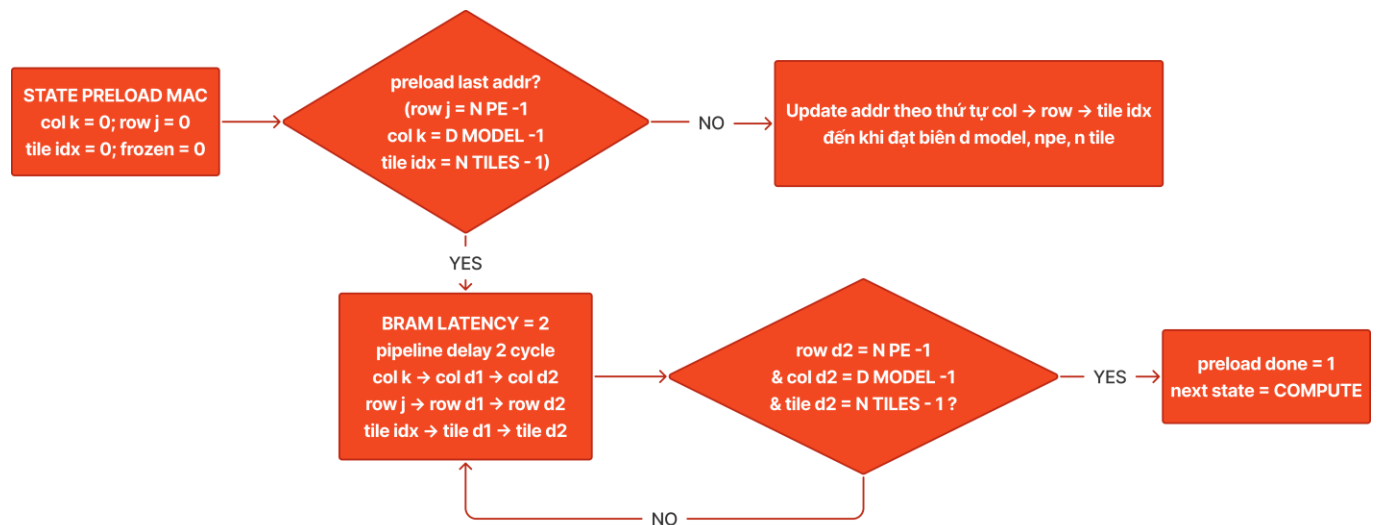
Phân tích từng giải thuật thực thi trong mỗi state sẽ được trình bày trong phần này.

Bắt đầu từ state IDLE, trạng thái tĩnh của IP, chỉ chuyển sang trạng thái tiếp theo là LOAD K khi có tín hiệu start từ control plane từ Zynq UltraScale+ MPSoC, thông qua giao tiếp AXI Lite.



Hình 4: Lưu đồ giải thuật của IP BRAM k ram

State LOAD K thực hiện nhận data của ma trận K từ DMA, thông qua giao tiếp AXI Stream, DMA sẽ thực hiện MM2S để lấy dữ liệu từ một IP BRAM thông qua một IP BRAM Controller, giao tiếp AXI. Kích thước thực thi là 16x64, mỗi phần tử có data width là 32, address width là 32, tín hiệu tlast assert khi truyền đủ length dữ liệu của ma trận K. Dữ liệu ma trận K sẽ được lưu vào một IP BRAM k_ram trong IP Linear để tính toán. Khi DMA đẩy data vào IP k ram thì địa chỉ sẽ được cập nhật liên tục mỗi cycle đến khi tlast assert, địa chỉ chỉ được cập nhật khi tín hiệu wea được assert, điều này chỉ xảy ra khi có cả cặp tín hiệu valid và ready assert. State này done khi k load done assert, điều này xảy ra khi có tín hiệu i s axis tlast assert.

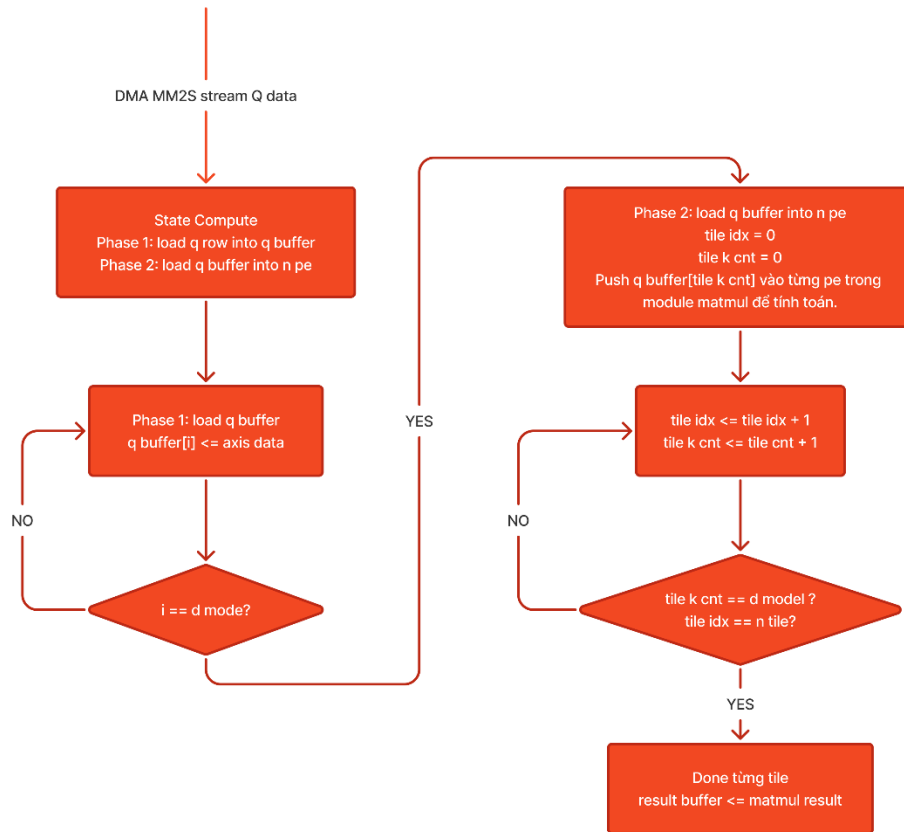


Hình 5: Lưu đồ giải thuật state Preload mac

State Preload Mac thực hiện quét và nạp dữ liệu từ IP BRAM k ram, dùng 3 bộ đếm từ inner ra ngoài là col, row, tile để duyệt từng phần tử rồi đưa vào từng pe. Do IP BRAM set up có latency là 2 nên có thêm delay 2 cycle.

- Preload col k ($0 \rightarrow D \text{ Model} - 1$): Duyệt từng cột của hàng K.
- Preload row j ($0 \rightarrow N \text{ Pe} - 1$): Khi hết 1 hàng K (col k chạm max) thì mới đổi sang pe tiếp theo trong cùng 1 tile.
- Preload tile idx ($0 \rightarrow N \text{ Tiles} - 1$): Khi hết cả N Pe hàng trong tile hiện tại thì mới đổi sang tile tiếp theo.

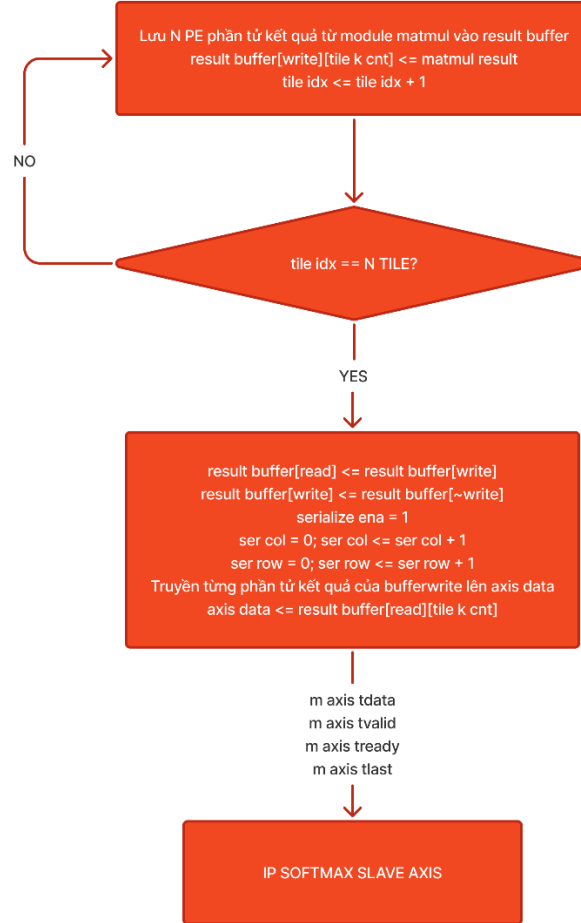
Ba bộ đếm tạo ra địa chỉ $addr = (tile \text{ idx} * N \text{ Pe} + row \text{ j}) * D \text{ Model} + col \text{ k}$. Khi last addr được assert, các bộ đếm delay 2 cycle đạt max thì preload done assert và chuyển sang next state là COMPUTE.



Hình 6: Lưu đồ giải thuật 1 của state COMPUTE

State COMPUTE thực hiện nhận từng data Q từ DMA MM2S vào và lưu trong q buffer rộng D model phần tử. Sau khi nạp xong, phase 2 thực hiện push q buffer vào từng tile của

submodule matmul, push đủ cho n tile, tất cả pe trong tất cả tile sẽ thực hiện nhân q buffer[tile k cnt] với hàng thứ i của K ram (Được lưu trong pe thứ i), kết quả được tích lũy, tiếp theo push lần lượt các phần tử còn lại của q buffer vào, cho đến khi đủ số phần tử 1 hàng là d model phần tử.



Hình 7: Lưu đồ giải thuật 2 của state COMPUTE

Khi 1 tile tính xong số pe của tile đó, kết quả sẽ push vào result buffer, quá trình này lặp lại đến khi số tile đạt max. Nghiên cứu sử dụng phương pháp double ping pong buffer, khi matmul tính xong 1 hàng sẽ lưu vào result buffer[write], khi truyền serialize ra sẽ stream result buffer[read] với $read \leq write$; $write \leq \sim write$, khi đó buffer write sẽ idle và tiếp tục thực hiện lưu kết quả từ row kế tiếp trong khi axis vẫn stream từng phần tử row hiện tại lên axis data đến IP Softmax. Khi truyền đủ seq len row của kết quả attention score thì tlast assert và next state là ST DONE.

1.3. Module matmul_ip

Module matmul_ip thực hiện phép nhân matmul bằng phương pháp SIMD broadcast, thay vì tính dot-product của một hàng ma trận A và một cột ma trận B tuần tự thì phương pháp này thực hiện lấy 1 phần tử của ma trận A nhân với từng hàng của ma trận B cùng một lúc. Số lượng N PE nếu bằng số hàng của ma trận K là N COLS thì số lượng tile là 1, nếu nhỏ hơn thì số tile được chia thành N TILES nhỏ, mỗi tile chứa N PE hoặc số pe nhỏ hơn N PE nếu không chia đều. Ý nghĩa của các thông số tham số hóa trong module matmul_ip như sau:

Table 1: Parameter module matmul_ip

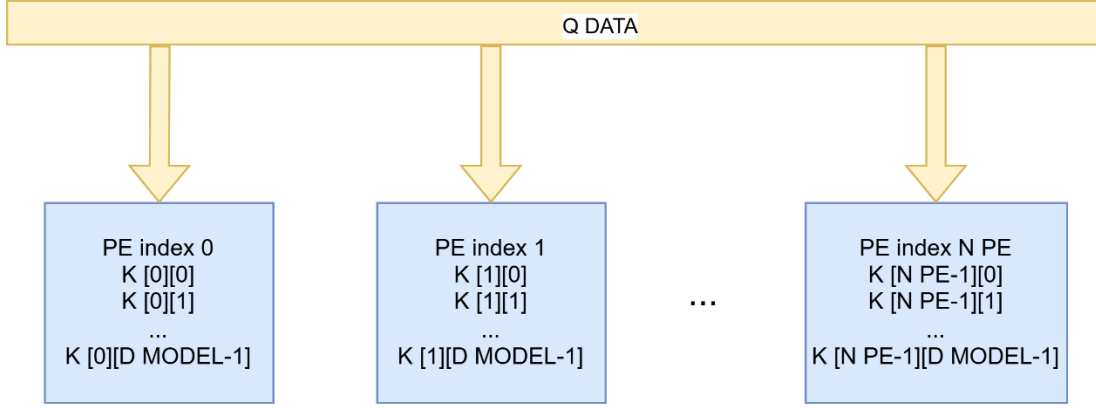
Tên	Ý nghĩa	Mặc định
N_COLS	Tổng số hàng của ma trận K	64
D_MODEL	Chiều dài vector dot-product (giống pe_unit)	64
N_PE	Số PE vật lý được instance song song	64
DATA_WIDTH	Độ rộng bit của dữ liệu	32
N_TILES	Số bank weight trong mỗi PE (D_HEAD + N_PE - 1) / N_PE	1

Table 2: Localparameter module matmul_ip

Tên	Giá trị	Ý nghĩa
J_W	$\$clog2(N_PE)$	Số bit cần để đánh chỉ số của 1 PE trong N_PE
C_W	$\$clog2(N_COLS)$	Số bit cần để đánh chỉ số của 1 cột trong N_COLS cột

Do N_PE có thể nhỏ hơn giá trị số hàng của ma trận K là N_COLS nên sẽ có 2 kịch bản có thể xảy ra.

Trường hợp	Điều kiện	Giá trị N_TILES
1	$N_PE = N_COLS$	$N_TILES = 1$
2	$1 \leq N_PE < N_COLS$	$N_TILES = (D_HEAD + N_PE - 1) / N_PE$

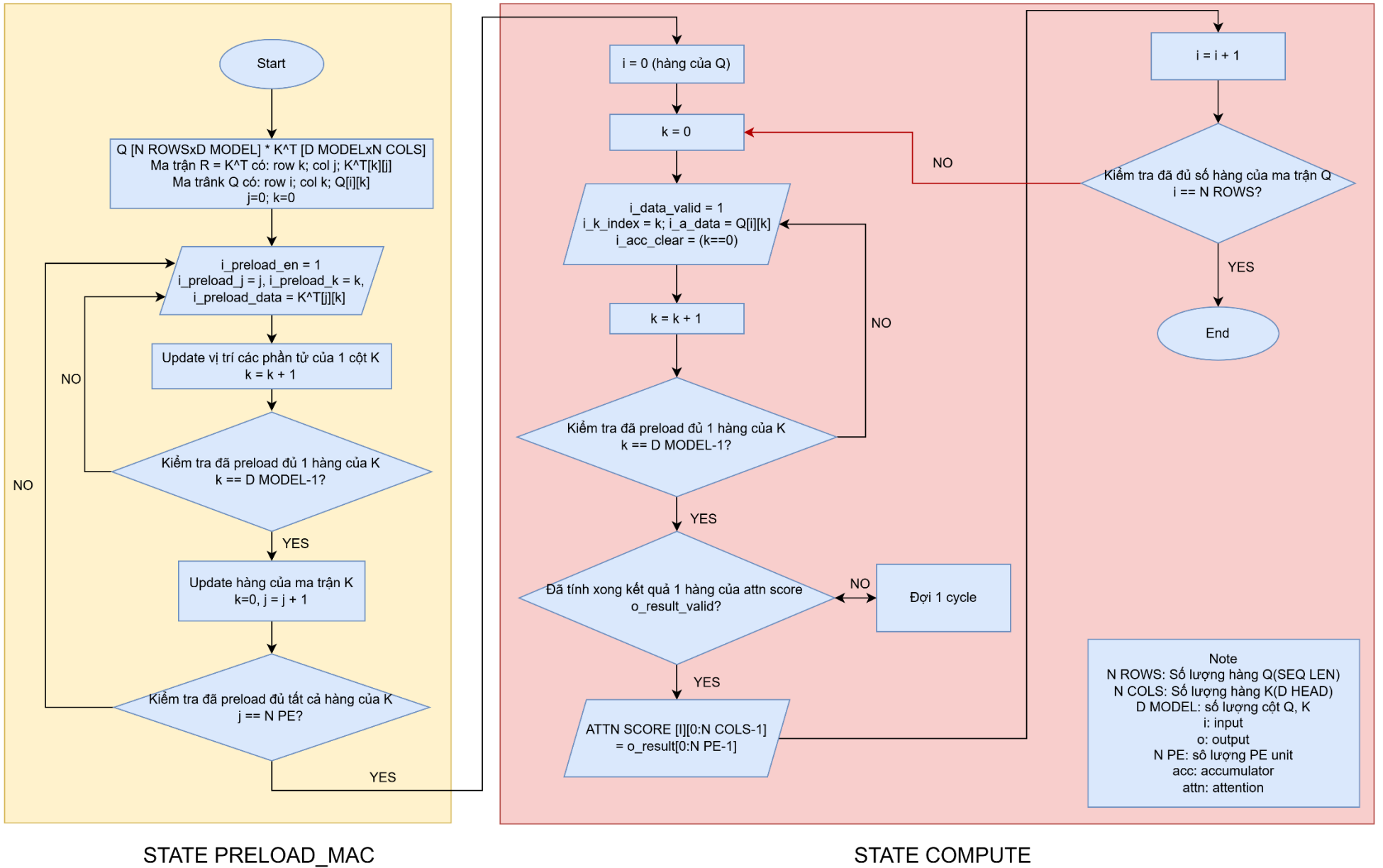


Hình 8: Sơ đồ cấu trúc nạp dữ liệu cho PE trong state COMPUTE với $N_TILES = 1$

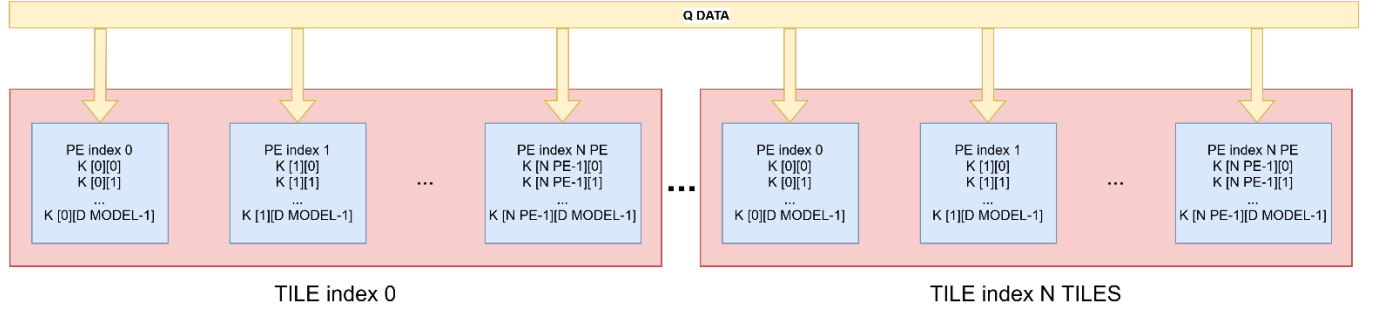
Case 1: $N_PE = N_COLS$ ($N_TILES = 1$)

Toàn bộ N_PE hoạt động như N_PE nhánh song song, cùng chia sẻ một bus dữ liệu đầu vào, tại state PRELOAD MAC, $i_preload_data$ (giá trị $K[j][k]$ tại hàng j , cột k) được broadcast tới tất cả N_PE instance, chỉ PE có index bằng $i_preload_j$ (hàng j của K) mới được ghi data vào buffer mảng weight, cụ thể $pe_preload_en = i_preload_en \& (i_preload_j == gp)$ cho phép tạo các bản $pe_preload_en$ riêng biệt. Sau N_PexD_MODEL cycle ghi tuần tự, mỗi PE lưu đúng 1 hàng của ma trận K (PE thứ j lưu tất cả phần tử của hàng thứ j của ma trận K).

Tại state COMPUTE, toàn bộ N_PE sẽ cùng nhận 1 giá trị $i_a_data = Q[i][k]$ (giá trị phần tử tại hàng i , cột k của Q) trong cùng 1 cycle. Bên trong mỗi PE, $weight_mux$ chọn ra $weight[i_tile_sel][i_k_index]$, tại case này $i_tile_sel=0$, phép nhân cộng tích lũy $i_a_data * weight_mux$ được lưu vào acc , qua D_MODEL cycle thì phép matmul sẽ tính xong phép nhân ma trận $Q * K^T$ của một hàng, kết quả được xuất ra o_result là array có kích thước D_MODEL .



Hình 9: Lưu đồ giải thuật của submodule *matmul_ip* trong 2 state *PRELOAD MAC* và *COMPUTE* của module linear trường hợp $N\text{ PE} = N\text{ COLS}$

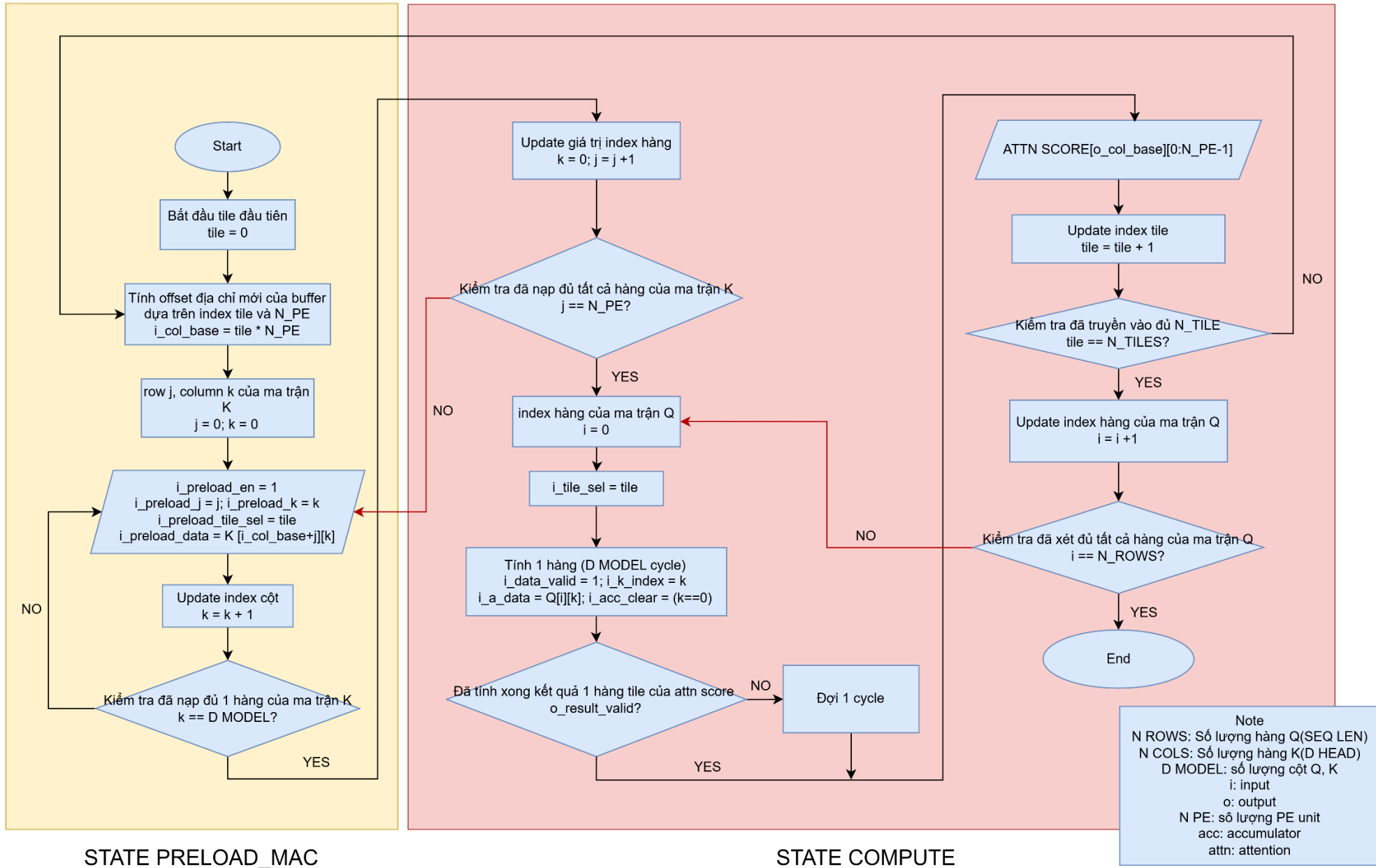


Hình 10: Sơ đồ cấu trúc nạp dữ liệu cho PE trong state COMPUTE với $N_TILES > 1$

Case 2: $N_PE < N_COLS$ ($N_TILES > 1$)

Khi N_PE nhỏ hơn N_COLS , số lượng PE vật lý không đủ để giữ toàn bộ số lượng N_COLS hàng của ma trận K cùng lúc. Thực hiện phân tách thành nhiều Tile khác nhau, mỗi tile chứa N_PE hoặc số lượng ít hơn, địa chỉ của mỗi data khi được ghi vào pe sẽ là hàng thứ $(tile * N_PE + j)$, cột thứ k , nghĩa là $i_preload_data = K[tile * N_PE + j; k]$, data được ghi vào đúng bank tile được chọn là $weight[i_preload_tile_sel]$. Datapath weight không bị ghi đè giữa các tile, cho phép preload trước nhiều tile rồi mới compute.

Tại state COMPUTE của module linear, tương tự case 1, data Q vẫn sẽ được đẩy vào các pe để tính matmul, nhưng sau D_MODEL cycle chỉ tính được N_PE pe, cần qua đúng $N_TILES \times D_MODEL$ cycle mới cho ra đúng kết quả một hàng của ma trận attention score.



Hình 11: Lưu đồ giải thuật của submodule `matmul_ip` trong 2 state `PRELOAD MAC` và `COMPUTE` của module `linear` trường hợp $N_{PE} < N_{COLS}$

1.4. Module pe_unit – Khối pe thực hiện nhân cộng tích lũy

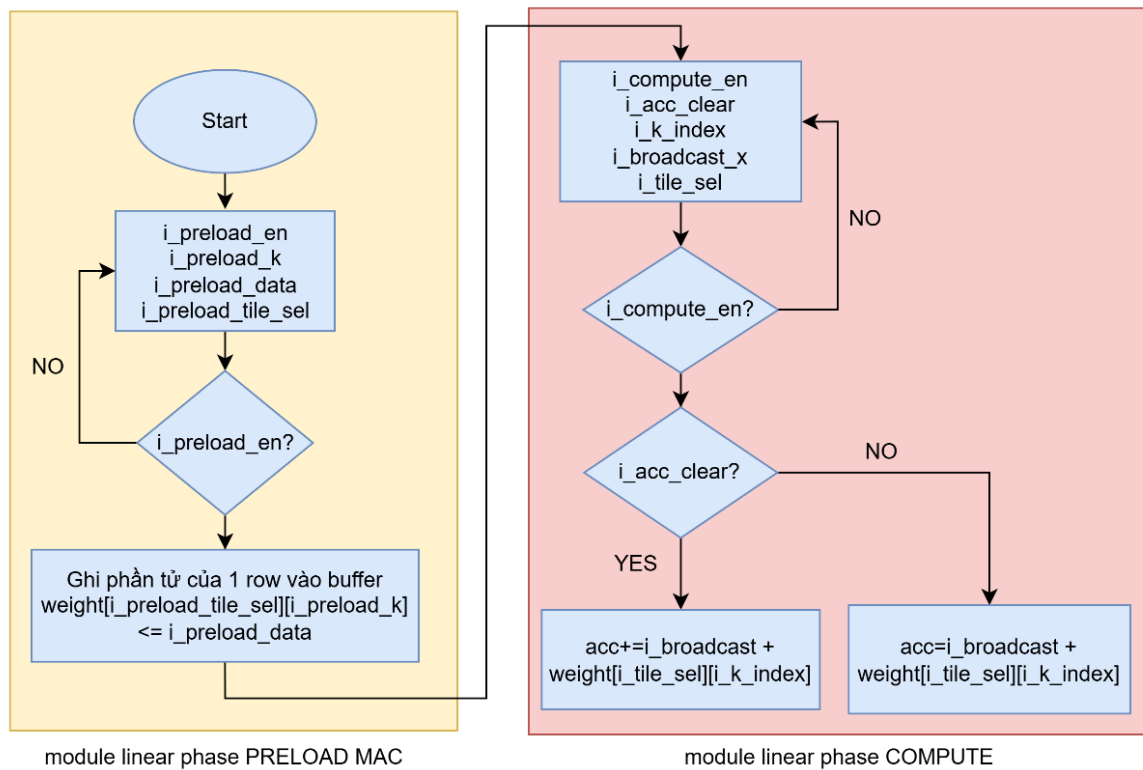
Module pe_unit thực hiện phép nhân cộng tích lũy, các thông số mặc định cho module như sau:

Table 3: Parameter module pe_unit

Tên	Ý nghĩa	Mặc định
D_MODEL	Số phần tử trong 1 vector	64
DATA_WIDTH	Độ rộng bit của mỗi số liệu vào/ra	16
N_TILES	Số bank weight độc lập lưu trong PE	1

Table 4: Localparameter module pe_unit

Tên	Giá trị	Ý nghĩa
ACC_WITDH	$DATA_WIDTH * 2 + \lceil \log_2(D_MODEL) \rceil$	Độ rộng accumulator
FRAC_BITS	$DATA_WIDTH / 2$	Số phần thập phân dùng khi truncate kết quả về lại DATA_WIDTH bit



Hình 12: Lưu đồ giải thuật phần mềm của module pe_unit

Submodule pe_unit hoạt động chủ yếu dựa trên 2 state chính của module linear là PRELOAD MAC và COMPUTE,

Trong state PRELOAD MAC, các tín hiệu input có ý nghĩa:

Table 5: Ý nghĩa các tín hiệu đầu vào của module pe_unit trong phase PRELOAD MAC của module linear

Tên	Ý nghĩa
i_preload_en	Tín hiệu enable cho phép nạp data 1 cột của K buffer vào 1 pe.
i_preload_k	Tín hiệu xác định vị trí của phần tử trong 1 cột.
i_preload_data	Giá trị weight của K buffer được nạp vào pe.
i_preload_tile_sel	Chọn tile (số lượng 0 đến N_TILES-1) để ghi weight.

- Khi có tín hiệu en, pe thực hiện lưu phần tử trong hàng của K matrix vào buffer hàng trong pe, địa chỉ được quyết định vào signal i_preload_tile_sel và i_preload_k, data là i_preload_data, buffer có kích thước N_TILES x D_MODEL.

Trong state COMPUTE của module linear, các tín hiệu input vào module pe_unit

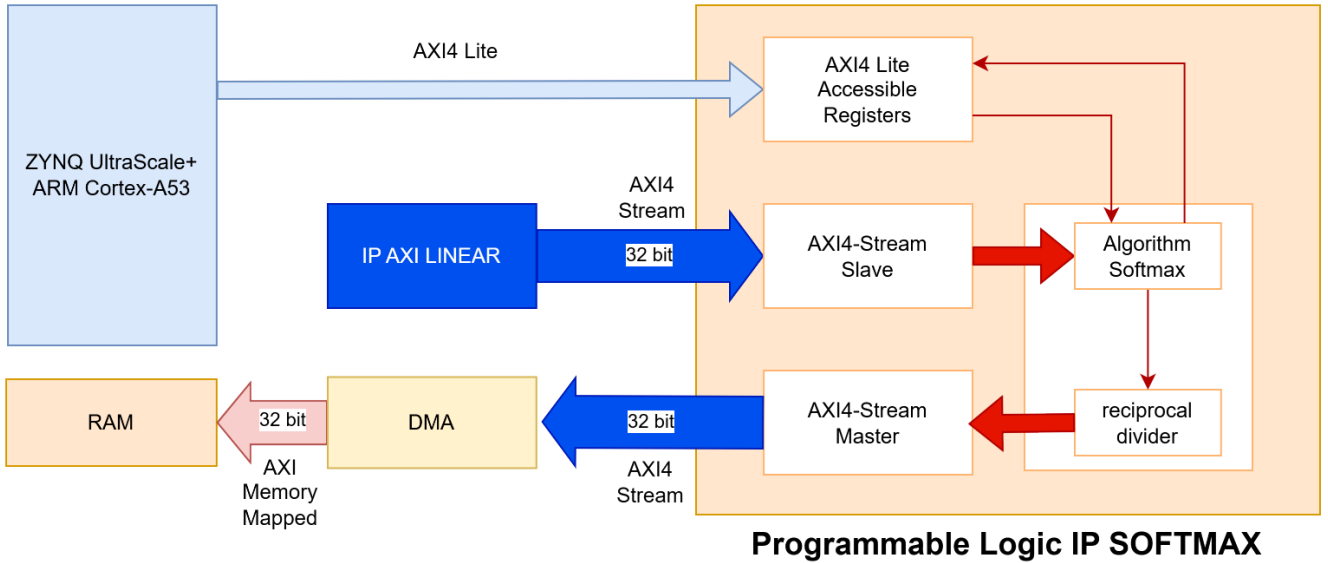
Table 6: Ý nghĩa các tín hiệu đầu vào của module pe_unit trong phase COMPUTE của module linear

Tên	Ý nghĩa
i_compute_en	Bật enable cho accumulator hoạt động.
i_acc_clear	Báo hiệu phần tử đầu tiên của 1 phép dot-product mới
i_k_index	Chỉ số cột của K matrix hiện tại, tương ứng là chọn weight trong tile.
i_broadcast_x	Giá trị đầu vào $Q[i][k]$ được nhân với weight.
i_tile_sel	Chọn tile (số lượng 0 đến N_TILES-1) để nhân với weight.

- Khi có tín hiệu i_compute_en, pe_unit sẽ xác định coi có phải là tín hiệu đầu tiên của 1 phép dot-product mới không, nếu phải thì sẽ lưu mới giá trị i_broadcast + weigh[i_tile_sel][i_k_index] vào acc, còn nếu không thì sẽ nhân cộng tích lũy vào biến acc với giá trị dot-product i_broadcast + weigh[i_tile_sel][i_k_index].

III. CẤU TRÚC IP SOFTMAX SCORE

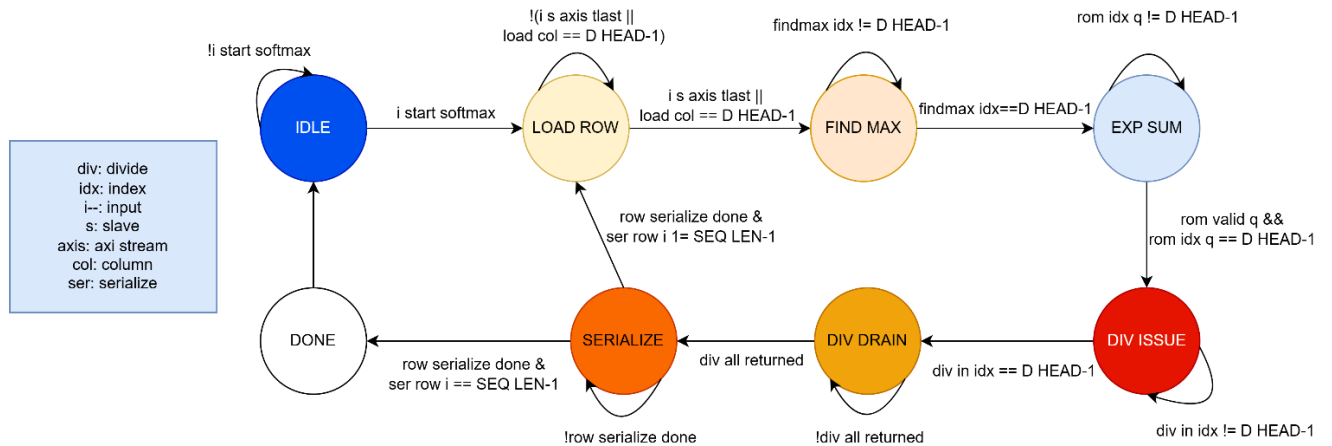
1. Cấu trúc IP AXI Softmax



Hình 13: Sơ đồ khối cấu trúc IP Softmax

IP Softmax gồm 2 phần, control plane và data plane. Control plane được SoC điều khiển với interface AXI Lite, data plane được kết nối AXI Stream với IP Linear và IP DMA.

1.1. FSM IP Softmax



Hình 14: FSM IP Softmax

Tham số:

Parameter	Giá trị đã test	Mô tả
D_HEAD	16	Số phần tử mỗi hàng

Parameter	Giá trị đã test	Mô tả
SEQ_LEN	16	Số hàng
DATA_WIDTH	16	Bit width input S (signed)
EXP_WIDTH	16	Bit width exp_rom / kết quả (unsigned)
RECIP_ADDR_W	12	Địa chỉ ROM reciprocal, không phụ thuộc D_HEAD/SEQ_LEN
RECIP_OUT_W	19	Output ROM reciprocal (Q0.19), ràng buộc $RECIP_OUT_W > EXP_WIDTH$

Output: Q1.15 unsigned, mỗi phần tử trong $[0, 1)$.

FSM 8 state: *IDLE* → *LOAD_ROW* → *FIND_MAX* → *EXP_SUM* → *DIV_ISSUE* → *DIV_DRAIN* → *SERIALIZE* → (loop hoặc *DONE*)

ST_IDLE

Trạng thái nghỉ. $o_busy = 0$. Chờ tín hiệu bắt đầu. Không thực hiện phép tính.

ST_LOAD_ROW

Nhiệm vụ: nạp D_HEAD phần tử của một hàng vào s_row_buf qua S_AXIS.

- $o_s_axis_tready = 1$ chỉ trong state này, đây là state duy nhất chấp nhận input.
- Mỗi beat hợp lệ ($row_transfer$) ghi $s_row_buf[load_col]$, tăng $load_col$.
- Kết thúc khi tlast hoặc $load_col == D_HEAD - 1 \rightarrow row_load_done = 1$, chuyển ST_FIND_MAX.

ST_FIND_MAX

Nhiệm vụ: tìm max_val của hàng vừa nạp, phục vụ trừ max trước exp (ổn định số học, tránh overflow exp), kỹ thuật chuẩn "softmax with max-subtraction".

- Duyệt tuần tự $s_row_buf[findmax_idx]$, 1 phần tử/cycle, so sánh running max.
- Kết thúc khi $findmax_idx == D_HEAD - 1 \rightarrow findmax_done = 1$, chuyển ST_EXP_SUM.
- Độ trễ: D_HEAD cycle.

ST_EXP_SUM

Nhiệm vụ: tính $\exp(x-max)$ cho từng phần tử qua ROM tra bảng, đồng thời tích lũy tổng sum_acc .

- Kiến trúc 2-stage pipeline: *scan_idx* phát địa chỉ ROM (comb ra *exp_rom_addr* từ *z_val*), *rom_idx_q/rom_valid_q* trễ 1 cycle khớp latency đọc ROM đồng bộ.
- Khi *rom_valid_q*: ghi *exp_row_buf[rom_idx_q]*, cộng dồn *sum_acc*.
- Kết thúc khi phần tử cuối (*rom_idx_q == D_HEAD-1*) trả về → chốt *sum_latched*, *expsum_done=1*.
- Độ trễ: *D_HEAD+1* cycle (do pipeline lệch 1 cycle giữa scan và accumulate).

ST_DIV_ISSUE

Nhiệm vụ: phát *D_HEAD* cặp (dividend, divisor) vào *reciprocal_divider* để tính *exp_row_buf[i] / sum_latched*.

- *div_dividend = exp_row_buf[div_in_idx] << 15* (scale Q1.15, đã giải thích ở câu trước).
- *div_divisor = sum_latched* cố định cho cả hàng (broadcast, không đổi theo *idx*).
- Issue 1 beat/cycle, dừng khi *div_in_idx == D_HEAD - 1*.

ST_DIV_DRAIN

Nhiệm vụ: chờ pipeline 3-cycle latency của *reciprocal_divider* trả hết kết quả còn tồn đọng sau khi đã issue xong.

- Thu kết quả qua *div_tvalid_out*, ghi tuần tự vào *out_row_buf[div_out_idx]* (in-order return, không cần tag matching vì latency cố định).
- Kết thúc khi *div_out_idx == D_HEAD-1* → *div_all_returned=1*, chuyển ST_SERIALIZE.
- Lưu ý: logic thu thập chạy song song cả trong ST_DIV_ISSUE lẫn ST_DIV_DRAIN (điều kiện chỉ check *div_tvalid_out*, không check state) — đúng vì kết quả có thể bắt đầu trả về ngay từ trong ST_DIV_ISSUE do pipeline 3-cycle, cần ST_DIV_DRAIN chỉ để "vét" các beat cuối chưa kịp trả trước khi issue xong.

ST_SERIALIZE

Nhiệm vụ: xuất *out_row_buf* ra *M_AXIS*, 1 phần tử/cycle khi *i_m_axis_tready=1*.

- Có backpressure: *serialize_active && ~tready* → counter đứng yên (*axis_out_stall*), tuân thủ AXI-Stream (data/valid phải giữ ổn định khi chưa được consume).
- Xong 1 hàng (*ser_col_j == D_HEAD - 1*) → nếu là hàng cuối (*ser_row_i == SEQ_LEN - 1*) thì chuyển ST_DONE, ngược lại quay lại ST_LOAD_ROW để xử lý hàng tiếp theo (dòng 249) — đây là vòng lặp ngoài theo *SEQ_LEN* hàng.

ST_DONE

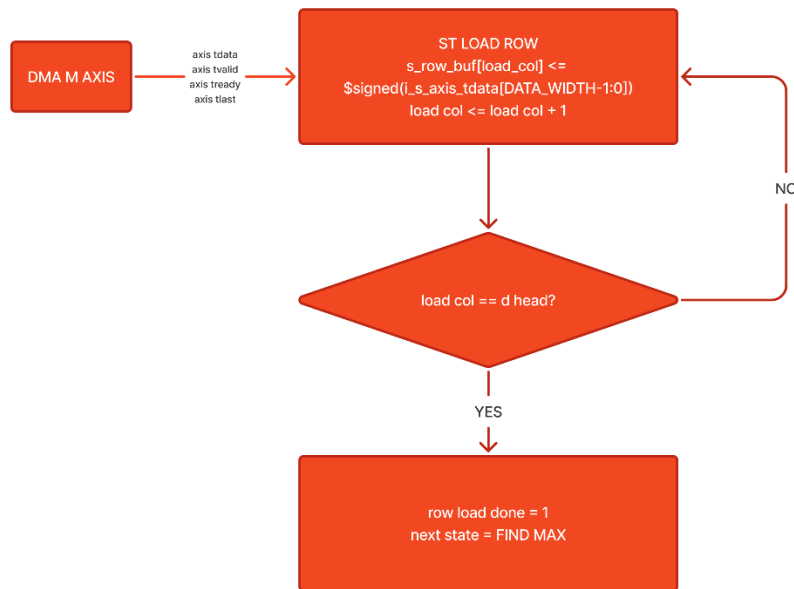
o_softmax_done = 1 (1 cycle), reset *ser_row_i*, quay về ST_IDLE.

Register map: 0x00 bit[0] = Start (tự clear khi busy rising); 0x04 bit[0] = Done (latched), bit[1] = Busy (tổ hợp). ROM phụ trợ: exp_rom.coe (2048 entry), recip_rom.coe (4096 entry) — cả hai là Block Memory Generator IP

1.2. Giải thuật thực thi FSM Softmax score

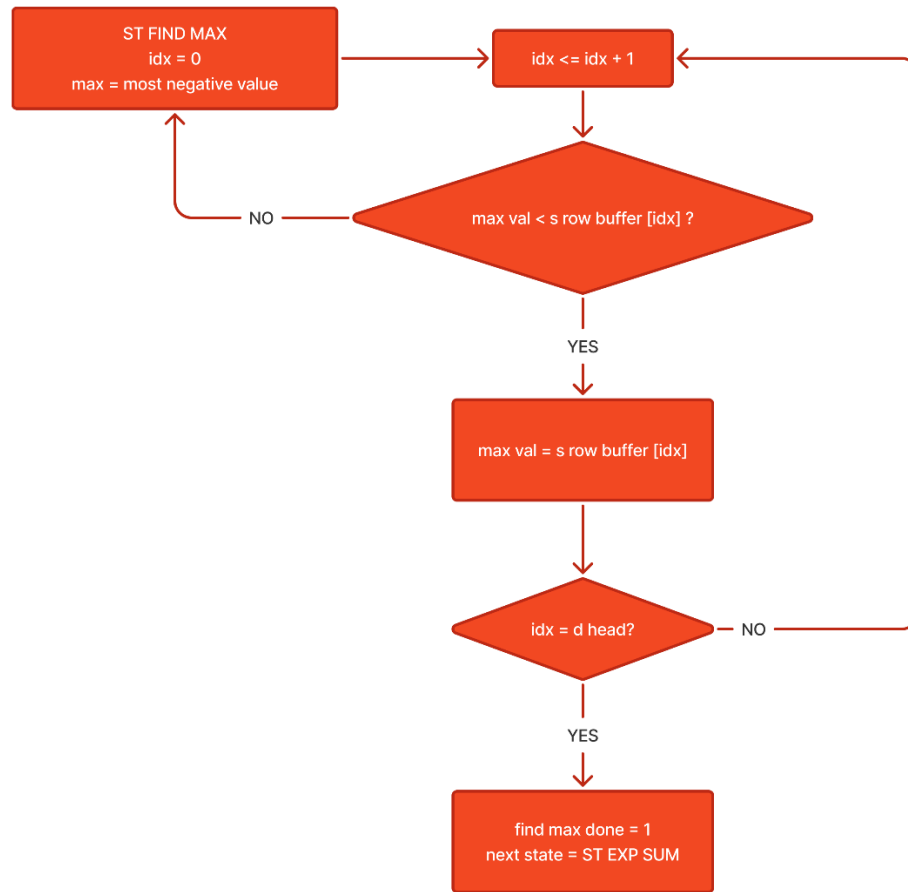
Phân tích từng giải thuật thực thi trong mỗi state sẽ được trình bày trong phần này.

Bắt đầu từ state IDLE, trạng thái tĩnh của IP, chỉ chuyển sang trạng thái tiếp theo là LOAD ROW khi có tín hiệu start từ control plane từ Zynq UltraScale+ MPSoC, thông qua giao tiếp AXI Lite.



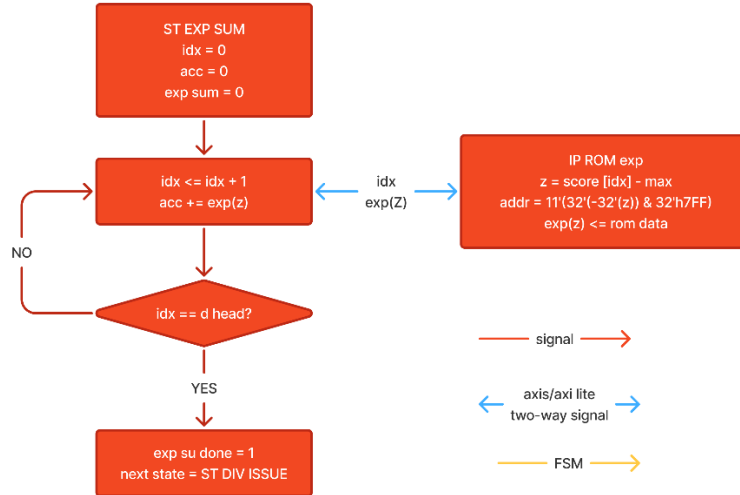
Hình 15: Lưu đồ giải thuật của FSM Load Row

State Load Row thực hiện load từng hàng của ma trận input vào cho hàm softmax, ma trận đó là attention score hay score, được lưu trong score buffer và được DMA MM2S truyền vào thông qua AXI Stream. Data truyền vào mỗi lần có số phần tử là số phần tử của 1 row trong score. Khi truyền đủ D Head phần tử thì load row done assert. Next state là ST FIND MAX.



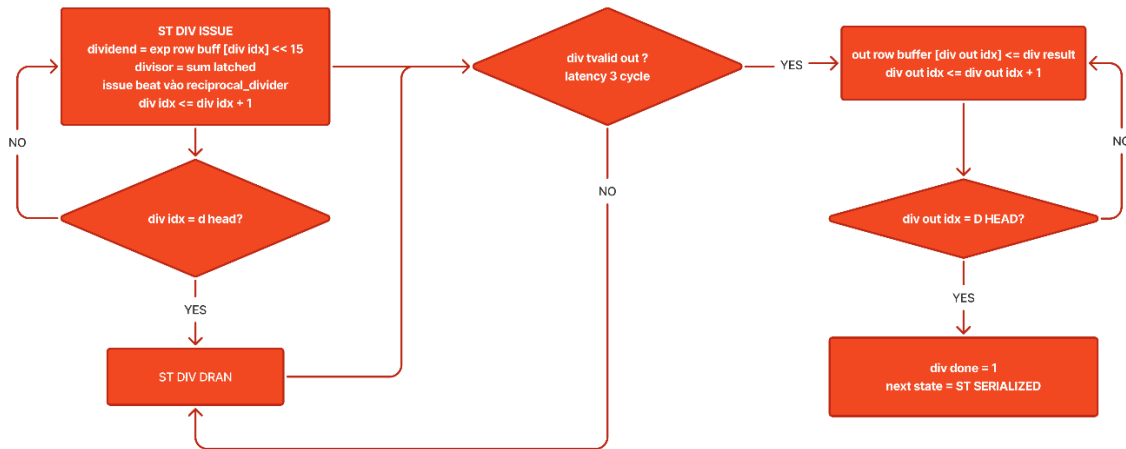
Hình 16: Lưu đồ giải thuật của FSM state Find Max

State Find Max thực hiện duyệt qua từng phần tử trong row để tìm phần tử có giá trị lớn nhất, quá trình duyệt dựa trên index trong array score buffer, khi phần tử được duyệt có giá trị lớn hơn giá trị max hiện tại thì sẽ update giá trị max. Khi duyệt đủ số lượng phần tử thì next state là ST EXP SUM.



Hình 17: Lưu đồ giải thuật của FSM Exp Sum

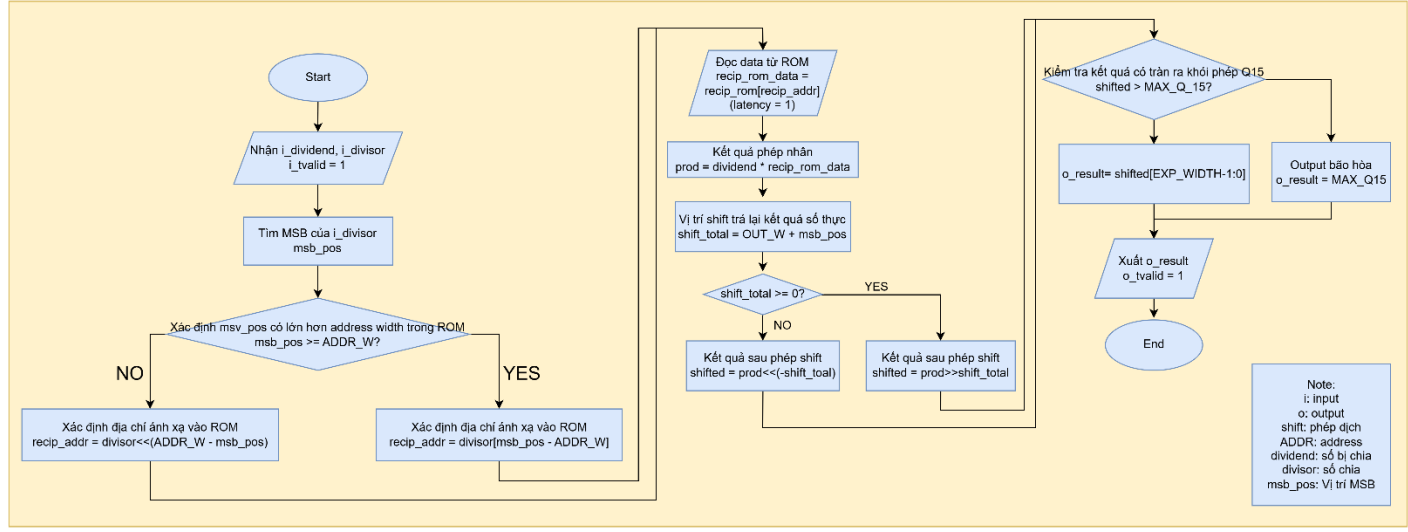
State Exp Sum thực hiện pipeline cho 3 nhiệm vụ chính, sử dụng phương pháp chuẩn hóa $z = x - \max(x)$, phương pháp LUT để tính hàm exp của z-tính tổng acc của $\exp(z)$. Sử dụng IP ROM là exp rom cho LUT, địa chỉ được encode là 11 bit cuối của $-z$ (do z có giá trị âm), địa chỉ này cho ra exp rom data (latency=1), chúng ta ghi giá trị exp này vào 1 buffer exp để dùng cho state sau. Tiếp theo, thực hiện acc cho giá trị data này, quá trình này kết thúc khi $\text{idx} = \text{d head}$ và exp sum done assert, next state là ST DIV ISSUE và ST DIV DRAIN.



Hình 18: Lưu đồ giải thuật FSM của ST DIV ISSUE và ST DIV DRAIN

ST DIV ISSUE và ST DIV DRAIN là hai state liên kế nhau thực thi nhiệm vụ phép chia $\frac{\exp(z)}{\text{sum}[\exp(z)]}$ và lưu kết quả trong out buffer. Trong Div Issue, dividend được assign là từng phần tử trong exp row buffer, divisor là sum của row đang xét, thực hiện truyền hai thông số này vào module reciprocal divider để thực hiện phép chia. Khi thực hiện đủ cho 1 row thì chuyển sang state tiếp theo là SERIALIZER.

1.3. Module reciprocal_divider



Hình 19: Lưu đồ giải thuật thực thi module reciprocal_divider

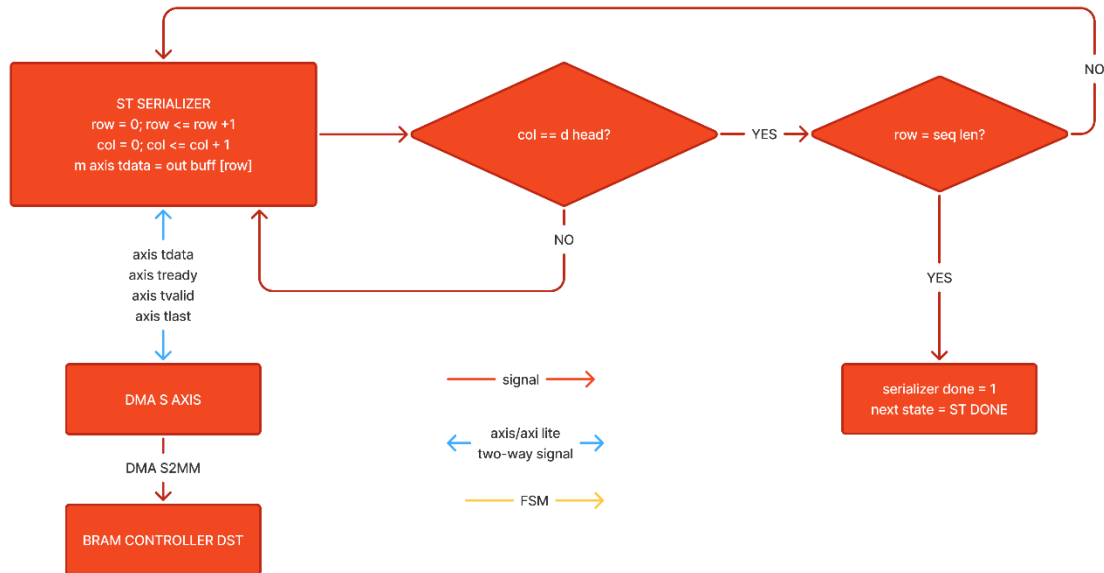
Trong thiết kế module reciprocal divider, kết quả chia được biểu diễn dạng Q1.15 unsigned, 1 bit ngầm định cho phần nguyên (luôn bằng 0 vì thương số $\frac{\exp}{\text{sum}}$ luôn nhỏ hơn 1). Sử dụng kỹ thuật range reduction tách divisor thành dạng mantissa nhân cho lũy thừa của msb, $\text{divisor} = \text{mantissa} * 2^{\text{msb}}$, $\text{mantissa} \in [1.0, 2.0)$, sau đó ta áp dụng phương pháp LUT cho mantissa trong khoảng $[1.0, 2.0)$ bằng IP ROM, cụ thể là lưu các giá trị trong ROM của biến $\frac{1}{\text{mantissa}}$. Module này được thực thi với kiến trúc pipeline 3 tầng:

- Stage 0 – Find MSB: Priority encoder duyệt divisor từ bit cao nhất xuống, xác định msb pos. Lấy ADDR W bit liền kề dưới MSB làm địa chỉ tra ROM, đây chính là phần fractional của mantissa sau khi đã chuẩn hóa (bit MSB=1 không cần lưu vì luôn ngầm định bằng 1).
- Stage 1 – ROM read (latency 1 cycle): Địa chỉ từ Stage 0 được đưa vào recip_rom (Block Memory Generator IP, nội dung sinh từ golden model offline), trả về giá trị $\frac{1}{\text{mantissa}}$ dạng Q0.19 unsigned. Khi đó, phép chia cần tính trở thành phép nhân.

$$\frac{\text{dividend}}{\text{divisor}} = \text{dividend} * \left(\frac{1}{\text{mantissa}} * 2^{-\text{msb}_{\text{pos}}} \right)$$

- Stage 2 – Multiply shift: Nhân $\text{dividend} \times \text{recip_rom_data}$, sau đó dịch phải $(\text{OUT_W} + \text{msb}_{\text{pos}})$ bit để khớp lại scale mong muốn (Q1.15), phép dịch phải có bao

gồm OUT_W là do trước đó giá trị $\frac{1}{mantissa}$ chỉ là giá trị được scale lên OUT_W lần, không phải giá trị thực.



Hình 20: Lưu đồ giải thuật của FSM SERIALIZER

ST SERIALIZER thực hiện quét từng phần tử trong out buffer, sau đó push lên axis tdata để truyền data ra ngoài, quá trình truyền kết thúc khi col = D HEAD và row = SEQ LEN, khi đó tlast được assert, IP Softmax hoàn thành nhiệm vụ tính toán Softmax score.

IV. MÔ PHỎNG VIVADO

1. Cấu trúc thực thi

Cấu trúc phần cứng khi mô phỏng:

Cấu trúc mô phỏng sẽ bao gồm các ip sau:

Table 7: Thông tin các IP được sử dụng trong testbench

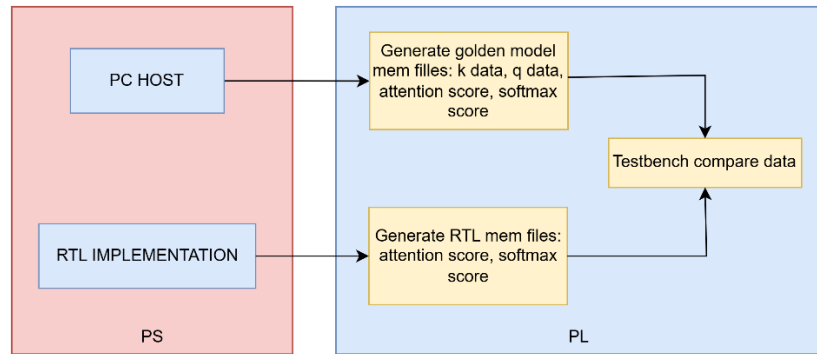
Tên IP	Type	Interface	Mô tả
AXI Verification IP	Master	AXI Full AXI Lite	Control plane của các IP DMA, Linear, Softmax.
Smart connect	Axi interconnect		Tạo các cổng slave, master để master và slave truyền, nhận lệnh.
DMA	Slave	AXI Lite AXI Stream	Được AXI VIP điều khiển init qua AXI Lite. Nhận data output từ IP Softmax qua AXIS.
	Master	AXI Stream	Push data K, Q qua AXIS tới IP Linear.
Linear	Slave	AXI Stream	Nhận K, Q từ DMA.
	Master	AXI Stream	Push Attention score tới Ip Softmax qua AXIS.
Softmax	Slave	AXI Stream	Nhận Attention Score qua AXIS.
	Master	AXI Stream	Push softmax score tới DMA qua AXIS.
BRAM Controller SRC		AXI	Controller ip BRAM SRC, DMA thực hiện MM2S để lấy data.
BRAM SRC			Lưu K, Q data
BRAM Controller DST		AXI	Controller ip BRAM DST, DMA thực hiện S2MM để ghi data.
BRAM DST			Lưu Softmax score data.

Địa chỉ và kích thước bộ nhớ cho các IP:

Table 8: Địa chỉ các IP được sử dụng trong testbench

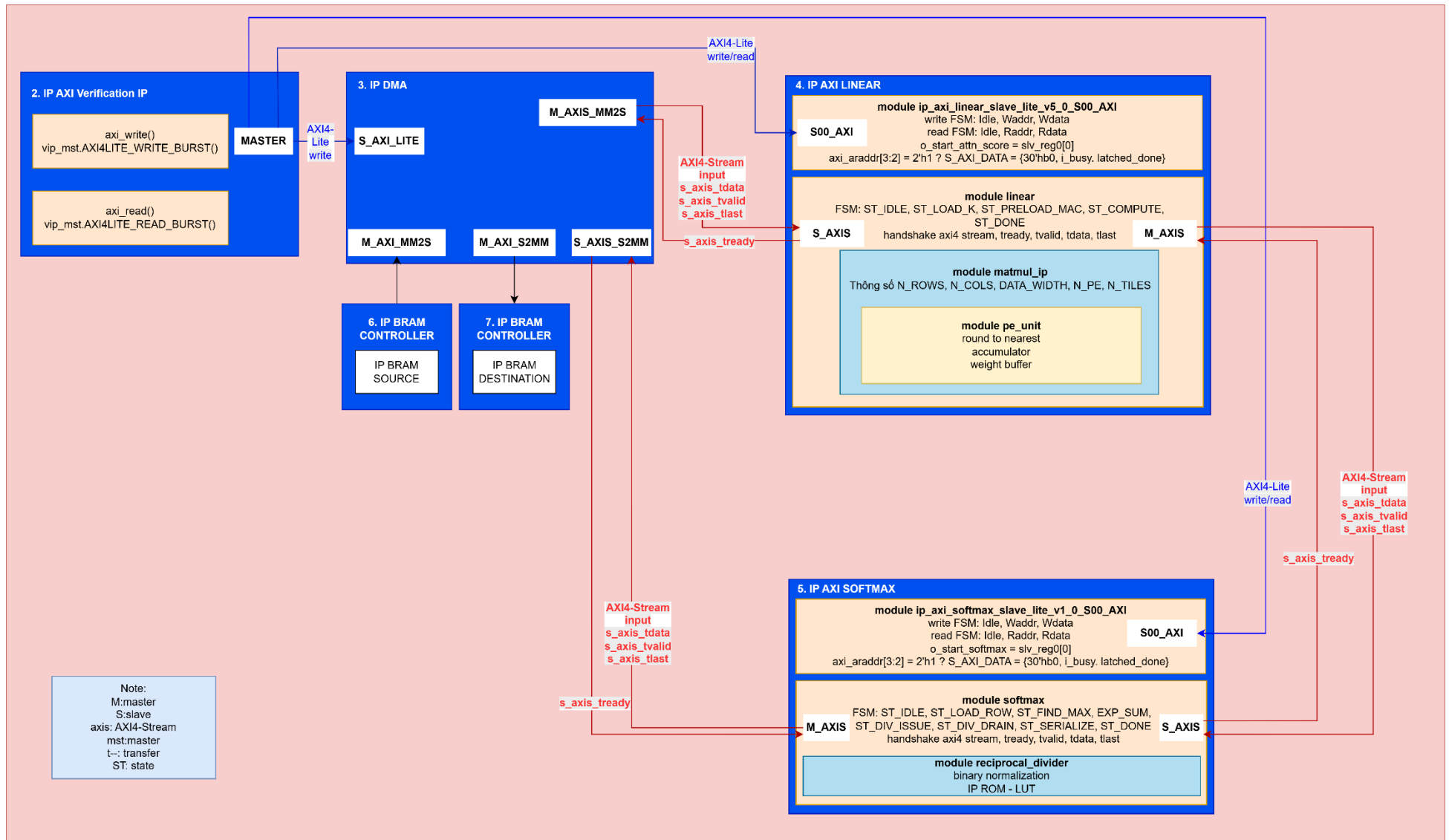
Tên IP	Địa chỉ	Kích thước
DMA	0x4000_0000	64K
Linear	0x4001_0000	64K
Softmax	0x4002_0000	64K
BRAM Controller SRC	0x0	32K
BRAM Controller DST	0x1_0000	16K

2. Cấu trúc mô phỏng



Hình 23: Sơ đồ PS lại PL cấu trúc mô phỏng

Cấu trúc mô phỏng chính sẽ là golden model sinh ra các file mem data input và các mem file golden cho softmax score và attention score, testbench sẽ thực hiện so sánh kết quả từ RTL và Golden mem file để đối chiếu.



Hình 24: Sơ đồ khối PL thực hiện testbench

Testbench thực hiện xác thực end-to-end pipeline `ip_axi_linear` → softmax bằng phương pháp so sánh với golden model, thông qua AXI VIP điều khiển DMA và IP dưới test. Flow gồm 4 giai đoạn tuần tự:

Giai đoạn 1 – Chuẩn bị dữ liệu (Init): Task `init_src_data()` đọc dữ liệu K, Q từ file `.mem` (dump từ golden model Python: `k_ram.mem`, `q_ram.mem`) và golden score kỳ vọng từ `golden_softmax.mem` vào mảng `golden_score[]` trong testbench, dùng làm reference cho bước so sánh cuối.

Giai đoạn 2 – Preload BRAM nguồn: Task `preload_bram_src()` ghi trực tiếp K, Q data vào BRAM src thông qua giao diện AXI-Lite của AXI VIP, tại địa chỉ cố định `K_BASE = SRC_BASE`, `Q_BASE = SRC_BASE + NUM_BYTES_K`. Đây là bước nạp sẵn dữ liệu vào memory-mapped source trước khi kích hoạt DMA, mô phỏng đúng luồng dữ liệu thực tế (`memory` → `DMA` → `IP`) thay vì đưa thẳng qua testbench.

Giai đoạn 3 – Vận hành DMA (2 sub-giai đoạn):

- `dma_reset()`: assert reset bit qua AXI VIP, đưa DMA về trạng thái sạch trước khi test, tránh trạng thái tồn dư từ lần chạy trước ảnh hưởng kết quả.
- `dma_start_transfer()`: chuỗi thao tác điều khiển 2 kênh DMA tuần tự
 1. Kênh MM2S (Memory-to-Stream) lần 1: set `MM2S_SA = K_BASE`, assert start signal của `ip_axi_linear`, set `length = Depth_K × 4` byte, chờ `wait_mm2s_done()`.
 2. Clear bit IOC (Interrupt-on-Complete) trong `DMA_SR`, re-assert start signal IP để chuẩn bị nhận beat tiếp theo.
 3. Kênh S2MM (Stream-to-Memory) đồng thời được cấu hình: `S2MM_DA = DST_BASE`, `length = Softmax_score_depth × 4` byte, nhận kết quả từ output stream của IP ghi ngược vào BRAM đích.
 4. Kênh MM2S lần 2: set `MM2S_SA = Q_BASE`, `length = Depth_Q × 4`, đẩy tiếp Q data vào IP.
 5. Chờ đồng thời `wait_mm2s_done()` và `wait_s2mm_done()`, đảm bảo cả luồng đẩy vào (K, Q) và luồng nhận kết quả ra (softmax score) đều hoàn tất trước khi sang bước so sánh.

Giai đoạn 4 – So sánh kết quả (Compare): Task `compare_bram_dst_with_golden_model()` đọc từng word từ `DST_BASE` qua AXI VIP (`axi_read`), so sánh với `golden_score[i]` đã nạp ở Giai đoạn 1. Với mỗi phần tử không khớp, in ra chỉ số, giá trị kỳ vọng, giá trị thực đo, và độ lệch (delta) để phục vụ debug, nếu toàn bộ khớp thì in PASS.

3. Kết quả mô phỏng

Thực hiện mô phỏng với bộ thông số đầu vào như sau:

Table 9: Thông số chuẩn hóa khi mô phỏng testbench

Parameter	Value
D MODEL	64
SEQ LEN	16
D HEAD	16
N PE	16
DATA WIDTH	16

```

init data src done
init src data time: 0.00 ns
preload data into bram src done
preload_bram_src time: 573770.00 ns
dma_reset time: 1320.00 ns
DEBUG: MM2S_DMASR right after reset = 0x00000001
1
2
3
4
mm2s done
5
6
7
8
9
10
11
mm2s done
s2mm done
dma transfer done
dma_start_transfer time: 46240.00 ns
PASS: all output words match golden model

```

```

00003192
000006f8
00001e03
00002970
00002902
000028b0
000017de
0000166d
0000260b
0000038b
0000528a
000003de
000014df
00003685
00001ff2
000014a8
00000d2b
000021b6
0000436c

```

Hình 25: Kết quả tcl của tb và giá trị rtl softmax được lưu trong mem file

Kết quả mô phỏng cho đúng softmax 16x16 phần tử giống với golden softmax score, kết quả waveform cũng cho các giá trị được push vào trong IP BRAM DST.

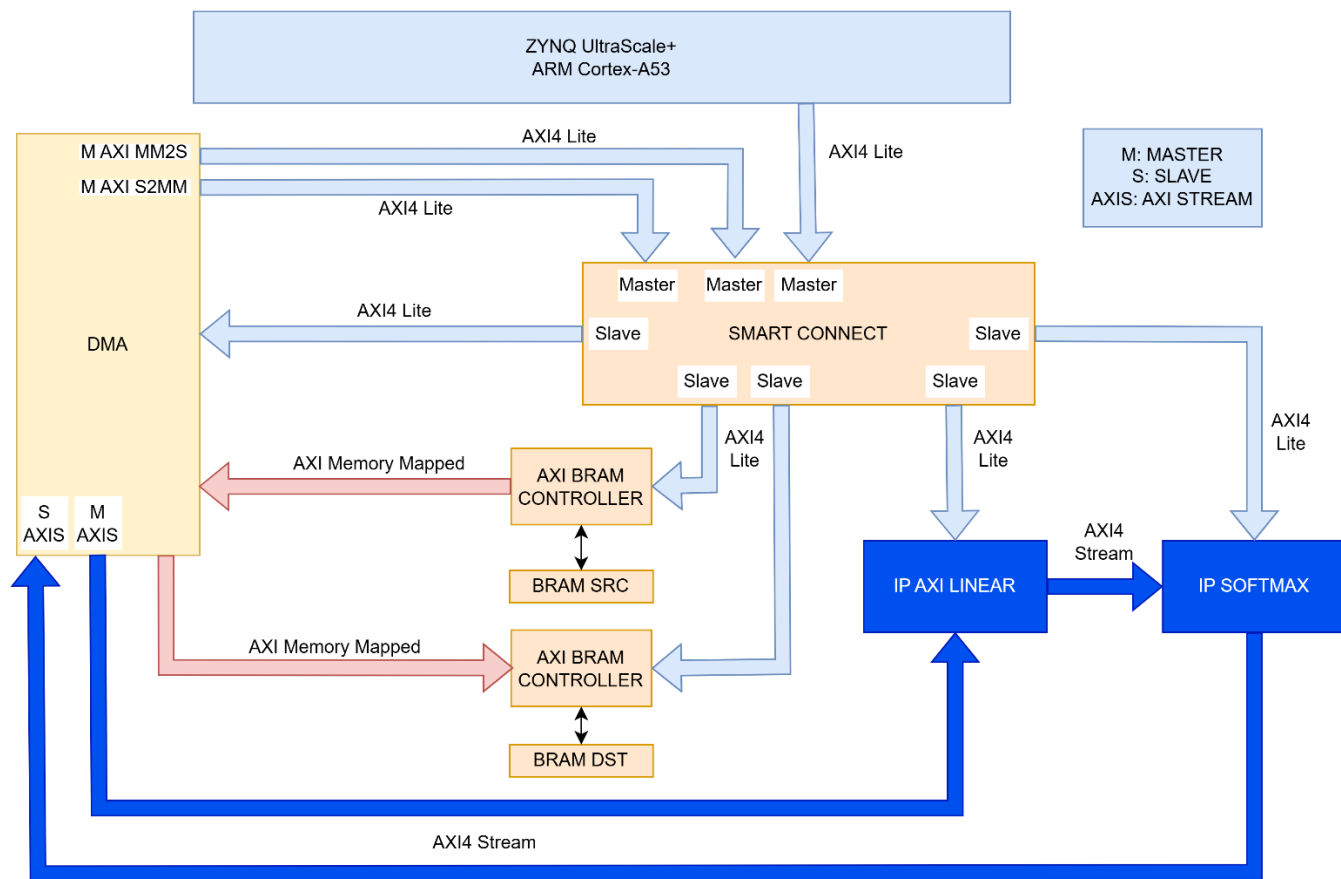


Hình 26: Softmax data được push vào BRAM DST từ DMA S2MM

V. THỰC HIỆN TRÊN KV260

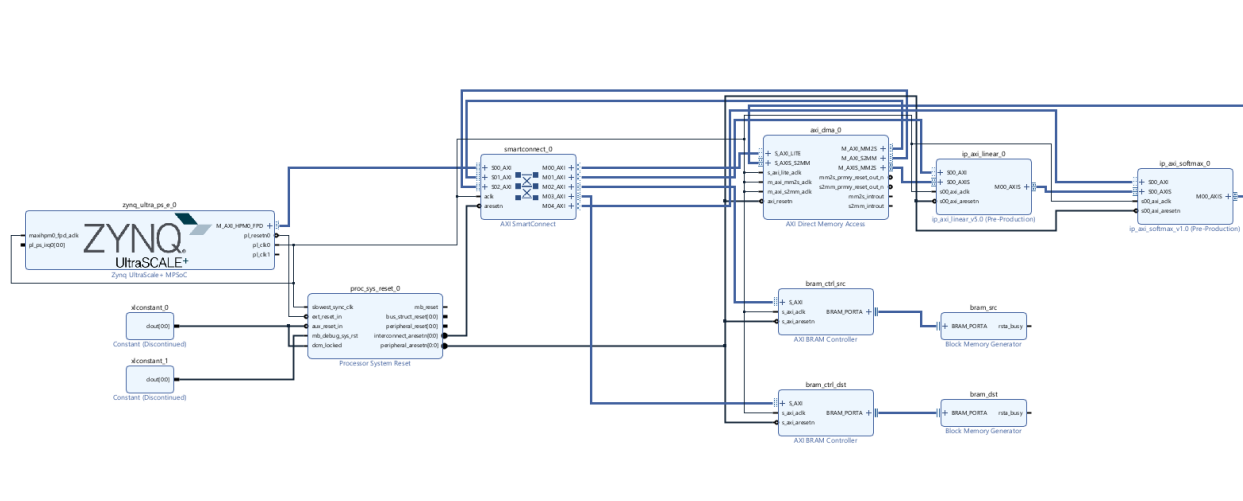
1. Cấu trúc thực thi

Cấu trúc phần cứng khi thực thi:



Hình 27: Sơ đồ khối cấu trúc phần cứng thực thi trên KV260

Cấu trúc Block Design:



Hình 28: Cấu trúc Block Design

Cấu trúc mô phỏng sẽ bao gồm các ip sau:

Table 10: Thông tin các IP được sử dụng khi thực thi trên KV260

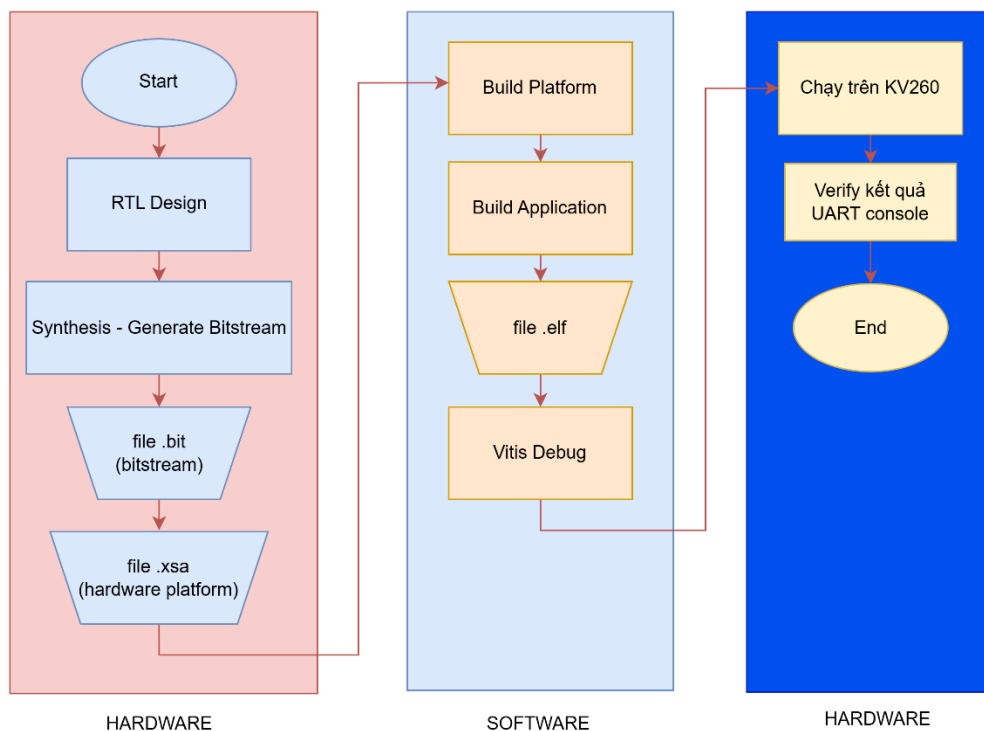
Tên IP	Type	Interface	Mô tả
Zynq UltraScale+ MPSoc	Master	AXI Full AXI Lite	Control plane của các IP DMA, Linear, Softmax.
Smart connect	Axi interconnect		Tạo các cổng slave, master để master và slave truyền, nhận lệnh.
DMA	Slave	AXI Lite AXI Stream	Được Zynq điều khiển init qua AXI Lite. Nhận data output từ IP Softmax qua AXIS.
	Master	AXI Stream	Push data K, Q qua AXIS tới IP Linear.
Linear	Slave	AXI Stream	Nhận K, Q từ DMA.
	Master	AXI Stream	Push Attention score tới Ip Softmax qua AXIS.
Softmax	Slave	AXI Stream	Nhận Attention Score qua AXIS.
	Master	AXI Stream	Push softmax score tới DMA qua AXIS.
BRAM Controller SRC		AXI	Controller ip BRAM SRC, DMA thực hiện MM2S để lấy data.
BRAM SRC			Lưu K, Q data
BRAM Controller DST		AXI	Controller ip BRAM DST, DMA thực hiện S2MM để ghi data.
BRAM DST			Lưu Softmax score data.

Địa chỉ và kích thước bộ nhớ cho các IP:

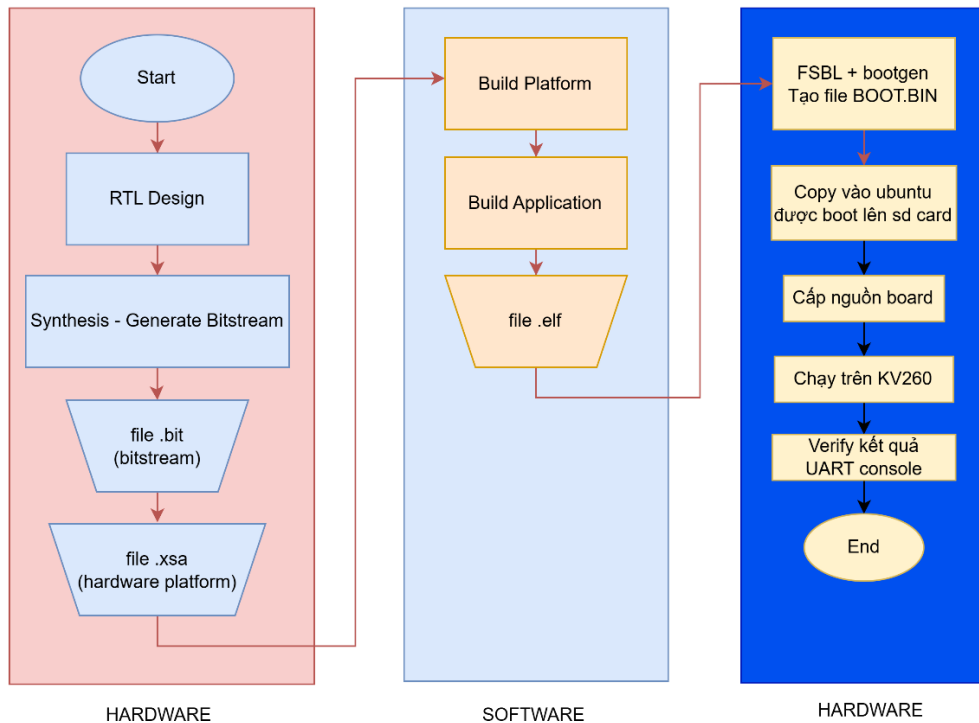
Table 11: Địa chỉ các IP được sử dụng khi thực thi trên KV260

Tên IP	Địa chỉ	Kích thước
DMA	0xA002_0000	64K
Linear	0xA003_0000	64K
Softmax	0xA004_0000	16K
BRAM Controller SRC	0xA000_0000	32K
BRAM Controller DST	0xA001_0000	16K

Sơ đồ khối luồng thực thi được chia ra thành hai phương pháp, phương pháp đầu tiên là boot dự án bằng JTAG, code C thông qua phần mềm Vitis IDE hay còn gọi là Baremetal, phương pháp còn lại là boot bằng linux ubuntu thông qua thẻ microSD:



Hình 29: Sơ đồ khối luồng thực thi bằng Baremetal



Hình 30: Sơ đồ khối luồng thực thi bằng linux thông qua SD card

2. Thông số thiết lập

Cấu trúc mô phỏng và flow hoàn toàn tương tự so với mô phỏng, chỉ thay đổi cpu thành zynq arm của kit kv260 và thực hiện đồ kit bằng phần mềm Vitis IDE, còn hiểu là bare metal.

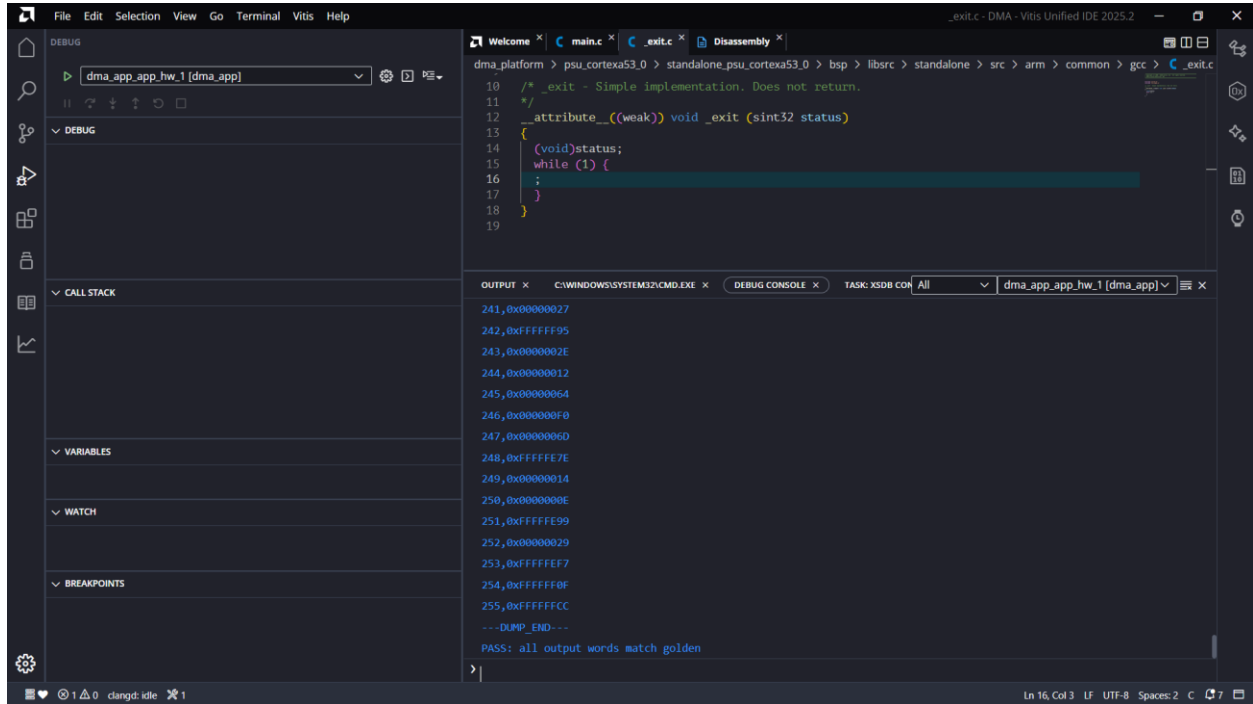
Thực hiện với bộ thông số đầu vào như sau:

Table 12: Thông số chuẩn hóa được sử dụng khi thực thi trên KV260

Parameter	Value
D MODEL	64
SEQ LEN	16
D HEAD	16
N PE	16
DATA WIDTH	16

3. Kết quả thực thi

3.1. Kết quả thực thi với phương pháp Baremetal



Hình 31: Kết quả tính toán attention score trên kv260

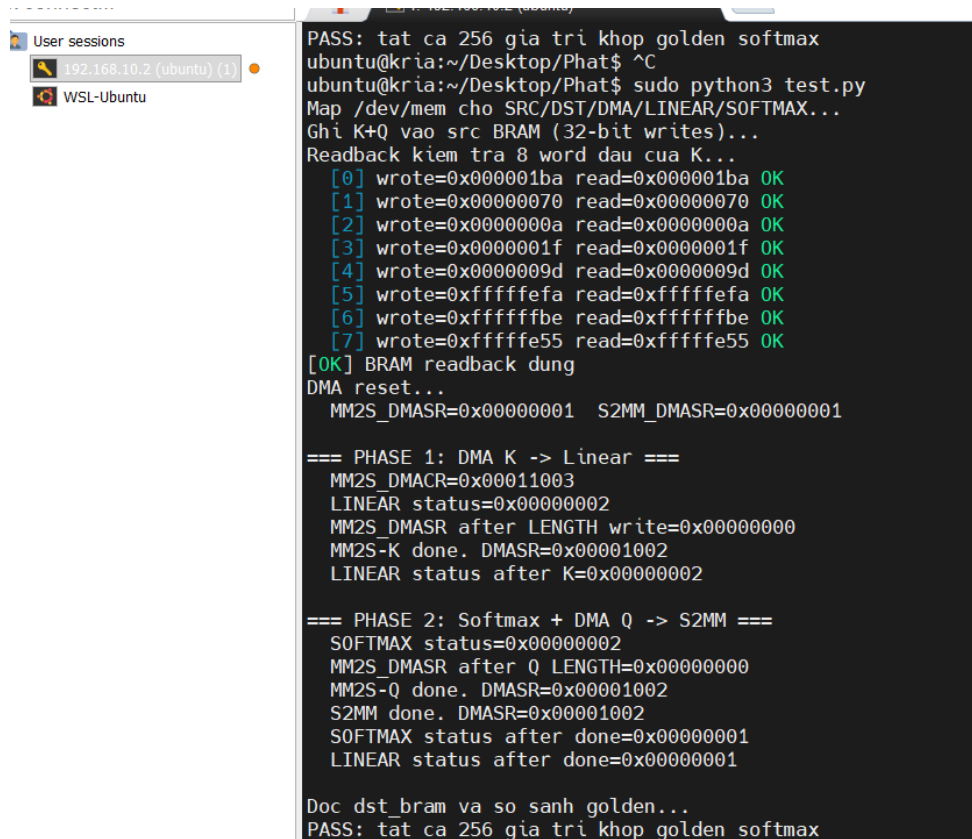
```
240,0x00000202
241,0x000009CD
242,0x00000587
243,0x00000A0D
244,0x0000090C
245,0x00000C65
246,0x00001571
247,0x00000CD1
248,0x000001D7
249,0x0000090C
250,0x000008E1
251,0x00000218
252,0x000009E2
2
53,0x00000304
254,0x00000344
255,0x000006DE
---DUMP_END---
PASS: all output words match golden softmax
```

Hình 32: Kết quả tính toán softmax score trên kv260

Kết quả mô phỏng đọc từ terminal output của Vitis IDE, nhận các kết quả softmax score và so sánh với golden model cho ra tất cả đều pass đối với 16x16 attention score và 16x16 softmax score.

3.2. Kết quả thực thi với Ubuntu Linux

Ngoài ra, bài nghiên cứu còn thực hiện push dự án lên trên kv260 bằng linux thông qua thẻ microsd, kết quả hoàn toàn đúng so với golden model.



```
PASS: tat ca 256 gia tri khop golden softmax
ubuntu@kria:~/Desktop/Phat$ ^C
ubuntu@kria:~/Desktop/Phat$ sudo python3 test.py
Map /dev/mem cho SRC/DST/DMA/LINEAR/SOFTMAX...
Ghi K+Q vao src BRAM (32-bit writes)...
Readback kiem tra 8 word dau cua K...
[0] wrote=0x000001ba read=0x000001ba OK
[1] wrote=0x00000070 read=0x00000070 OK
[2] wrote=0x0000000a read=0x0000000a OK
[3] wrote=0x0000001f read=0x0000001f OK
[4] wrote=0x0000009d read=0x0000009d OK
[5] wrote=0xfffffffffa read=0xfffffffffa OK
[6] wrote=0xffffffffbe read=0xffffffffbe OK
[7] wrote=0xffffffff55 read=0xffffffff55 OK
[OK] BRAM readback dung
DMA reset...
MM2S_DMASR=0x00000001 S2MM_DMASR=0x00000001

=== PHASE 1: DMA K -> Linear ===
MM2S_DMACR=0x00011003
LINEAR status=0x00000002
MM2S_DMASR after LENGTH write=0x00000000
MM2S-K done. DMASR=0x00001002
LINEAR status after K=0x00000002

=== PHASE 2: Softmax + DMA Q -> S2MM ===
SOFTMAX status=0x00000002
MM2S_DMASR after Q LENGTH=0x00000000
MM2S-Q done. DMASR=0x00001002
S2MM done. DMASR=0x00001002
SOFTMAX status after done=0x00000001
LINEAR status after done=0x00000001

Doc dst_bram va so sanh golden...
PASS: tat ca 256 gia tri khop golden softmax
```

Hình 33: Kết quả push dự án bằng linux

VI. TÀI LIỆU THAM KHẢO

- [1] ARM Limited, "AMBA AXI4-Lite Interface Specification," ARM IHI0022L, 2022.
- [2] ARM Limited, "AMBA AXI4-Stream Protocol Specification," ARM IHI0051B, 2021.
- [3] Xilinx Inc., "AXI DMA v7.1 Product Guide," PG021, AMD/Xilinx, 2022.
- [4] Xilinx Inc., "AXI Reference Guide," UG761, AMD/Xilinx, 2012.
- [5] R. Sass and A. G. Schmidt, The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000 All Programmable SoC, Strathclyde Academic Media, 2014.
- [6] Xilinx Inc., "Zynq-7000 SoC: Embedded Design Tutorial — IP Creation with Vivado," AMD/Xilinx, 2016.

VII. PHỤ LỤC

Dự án được lưu trữ tại github: [PHATVP22VT \(Arthur-Artoria\)](https://github.com/Arthur-Artoria/PHATVP22VT)